

Opracowanie z przedmiotu Narzędzia Programowania Rozproszonego '20 (once again)

[PDF Z ZMonitor withport: \[12346\] has entered the critical sectionadaniami
\(2K19\)](#)

[Merged](#)

[Zadanka z ADY i Javy](#)

[Jedzący filozofowie](#)

Spis treści

Spis treści	2
RPC	6
Schemat interakcji klient-serwer	6
Zagadnienia projektowe (problemy projektowe)	6
Przezroczystość dostępu	7
Warstwy protokołu RPC (ale niepotrzebne)	7
Przekazywanie parametrów	8
Konwersja danych (reprezentacja kanoniczna)	8
Konwersja bezpośrednia	9
Problem przetaczania – przykład złożonej struktury danych	10
Gwarancja wykonania (semantyka błędu) – ważne	10
Zagadnienia realizacyjne	12
Wiązanie dynamiczne klienta z serwerem	13
Protokoły warstwy RPC	14
Protokół BLAST	14
Protokół CHAN	15
Realizacja semantyki błędu	18
Usuwanie osieroconych obliczeń (sposoby)	19
Eksterminacja	19
Reinkarnacja	19
Łagodna reinkarnacja (mniej istotne)	19
Wygaśnięcie (Zwrócił na to szczególną uwagę, gdyż jest to rozwiązanie idealne dla protokołu próbkowania)	19
Protokół SELECT	19
Sun RPC/XDR (RPCGEN)	21
Właściwości SUN RPC:	21
Tworzenie aplikacji	21
Specyfikacja procedur zdalnych	22
Dostępne zakresy numerów programów	22
Przykładowy interfejs programu (usunięcie procesu)	23
Przykładowy interfejs programu (z wykorzystaniem struktury)	23
Definicje struktur w XDR oraz C	24

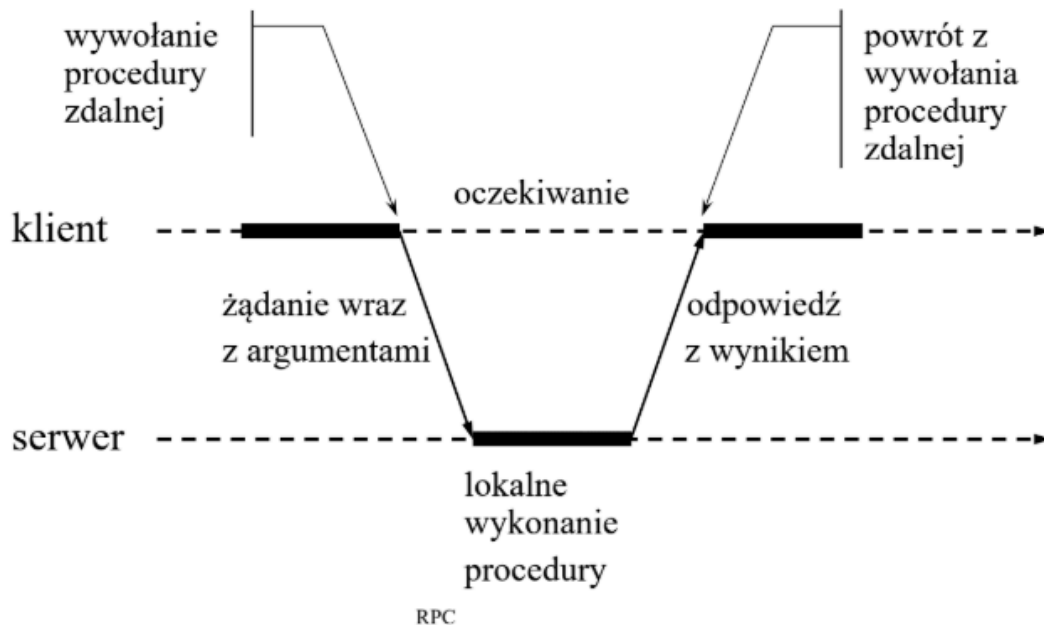
Standard XDR	26
Potok XDR	26
Filtr XDR	26
Potoki i filtry XDR	27
Filtry proste	27
Filtry złożone	27
Ogólna postać filtru złożonego	28
Filtry złożone do konwersji tablic (aczkolwiek raczej niepotrzebne)	28
Filtry złożone do konwersji łańcuchów znaków	29
Filtry złożone do konwersji struktur wskaźnikowych	29
Przykład filtru pochodnego	30
Zarządzanie pamięcią	30
Tworzenie potoków	30
Potoki komunikatów	31
Wywołanie metod zdalnych	32
Model systemu	32
Obiekt zdalny w środowisku Java	32
Dostępność obiektu	33
Przekazywanie obiektów jako parametrów	33
Przekazywanie przez wartość – przykład w Java RMI	34
Przekazywanie przez referencję	35
Przekazywanie przez kopiowanie i odtwarzanie	36
Architektura RMI	36
Hierarchia klas w definicji zdalnego obiektu	37
Tworzenie aplikacji rozproszonej w środowisku java RMI	38
Komunikacja między klientem a serwerem	39
Interfejs java.rmi.Remote	40
Zdalne udostępnianie obiektu (serwer)	40
Utworzenie i rejestracja zdalnego obiektu (strona serwera)	41
Zdalne wywołanie metody (strona klienta)	41
Lokalizowanie zdalnych obiektów	42
ØMQ (ZeroMQ)	43
Ogólny schemat komunikacji	43
Kontekst	43

Tworzenie gniazda	44
Wiązanie i łączenie gniazd	44
JMS	45
Cechy MOM	45
Koncepcja komunikacji oparta na kolejkowaniu	45
Paradygmat Publish/Subscribe	45
Model systemu typu MOM – podstawowe pojęcia	46
Funkcjonowanie systemu kolejkowania komunikatów	46
Warianty komunikacji	47
Komponenty JMS	48
Funkcjonalność JMS	48
Architektura JMS	49
Model punkt-punkt	49
Model subskrypcji	50
Model odbioru (konsumpcji) wiadomości	50
Model programistyczny JMS	51
Fabryki połączeń	51
Destinations	52
Sesje	53
***** Od Roberta *****	54
2020-05-05	54
Zdalne wywołania procedur - RPC	54
przekazywanie parametrów/wartości	54
gwarancje wykonania	54
wiązanie klient-serwer	55
procedura wiązania	55
warianty użycia RPC	55
2020-05-07	55
RPC - wiązanie dynamiczne klienta z serwerem	55
protokoły obsługi komunikacji:	55
realizacja semantyki błędów	56
usuwanie osieroconych obliczeń	56
Sun RPC/XDR	56

2020-05-12	57
Standard XDR	57
2020-05-14	59
Java RMI	59
2020-05-21	60
Ada :(	60
2020-06-02	61
Polyorb - object request broker	62
2020-06-16	62
kolejkowanie komunikatów i przestrzeń krotek	62
JMS	62
point-to-point	62
pub-sub	62
niezawodność komunikacji poprzez	63
przestrzeń krotek (default Linda)	63
Javaspaces	64
Java – ZADANIA	65
Problem Konsumenta–Producenta (elementarna synchronizacja wątków #synchronized)	65
Producent–Konsument (Lock)	67
ADA	68
Semafor ogólny	68
Semafor uogólniony z przepustowością	70
Bariera	73

RPC

Schemat interakcji klient-serwer



8

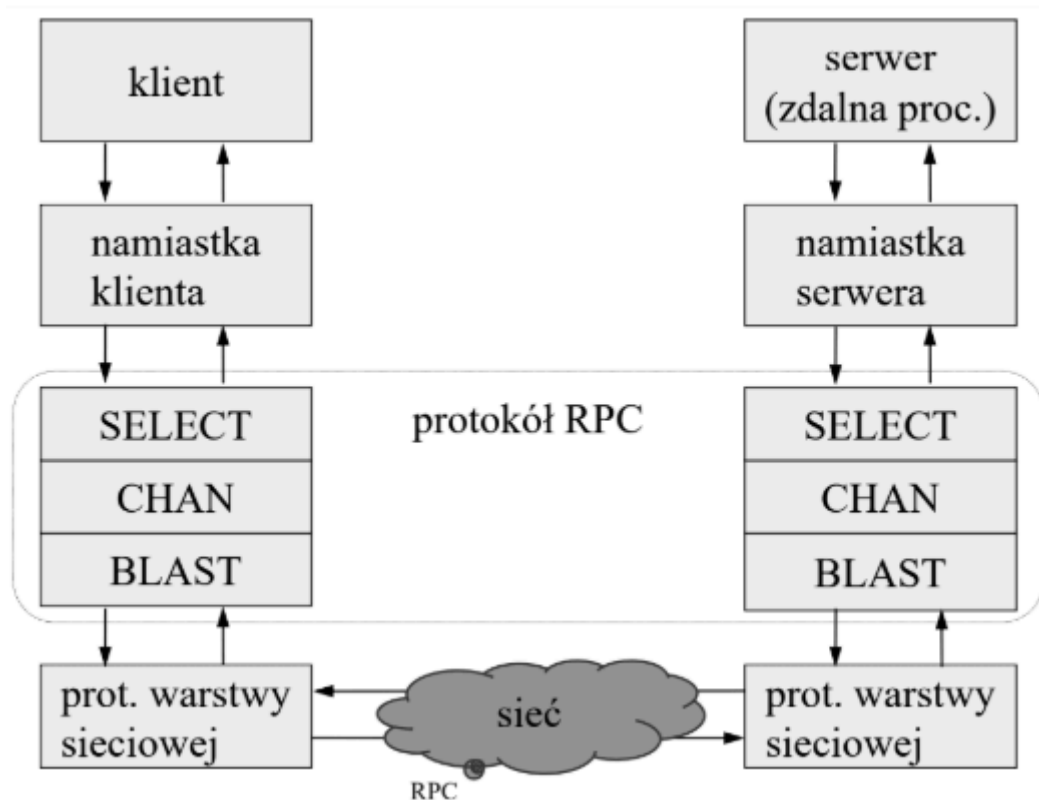
Zagadnienia projektowe (problemy projektowe)

- przeźroczystość dostępu — ukrycie komunikacji sieciowej przed aplikacją przez odpowiednie opakowanie funkcji komunikacyjnych namiastką klienta oraz serwera.
- gwarancja wykonania — ukrywanie błędów komunikacyjnych
- specyfikacja interfejsu — sposób opisu sygnatur procedur zdalnych (nazwy, typy parametrów)
- obsługa sytuacji wyjątkowych

Przezroczystość dostępu

- namiastka klienta (ang. client stub) — udostępnienie aplikacji klienckiej procedury lokalnej odpowiedzialnej za przesłanie danych do serwera oraz odebranie wyników
- namiastka serwera (ang. server stub) — udostępnienie aplikacji po stronie serwera procedury lokalnej odpowiedzialnej za odebranie identyfikatora procedury zdalnej do wywołania, parametrów procedury, a odesłanie wyników lub zgłoszenie wyjątków

Warstwy protokołu RPC (ale niepotrzebne)



Warstwy protokołu RPC to coś stosunkowo prymitywnego – rozważane są 3 warstwy:

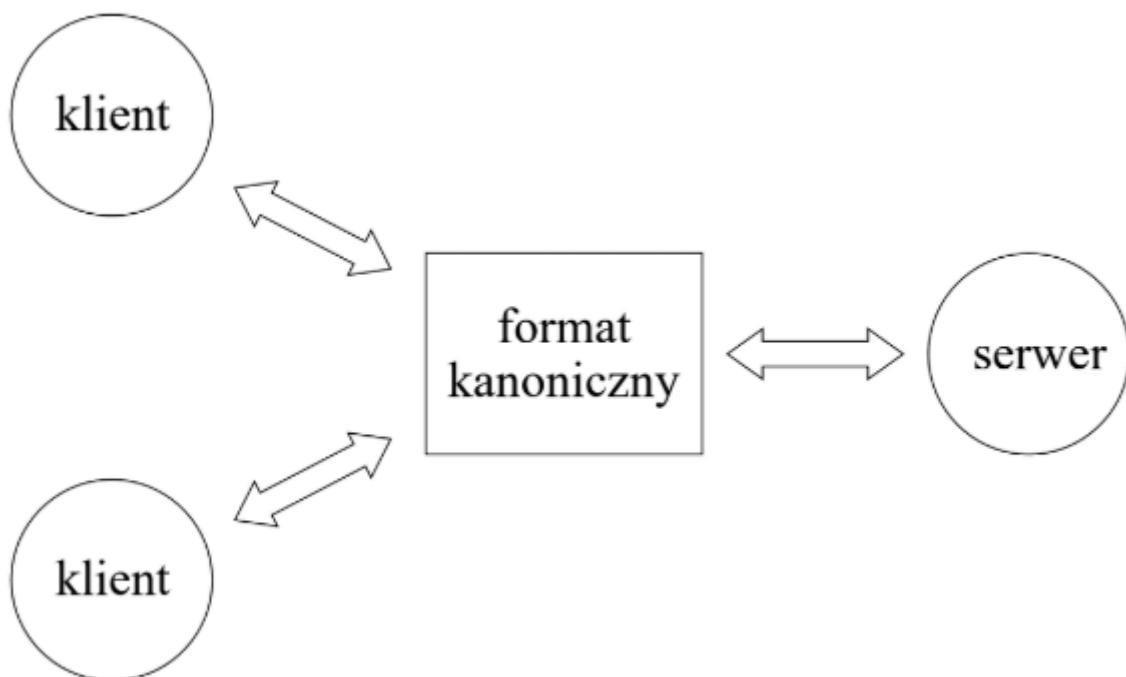
- BLAST (z punktu widzenia RPC nas mało interesuje),
- CHAN (warstwa dosyć specyficzna, rozważano tego typu protokół jako kolejny protokół warstwy transportowej),

- SELECT (identyfikacja procedur, jak identyfikować).

Przekazywanie parametrów

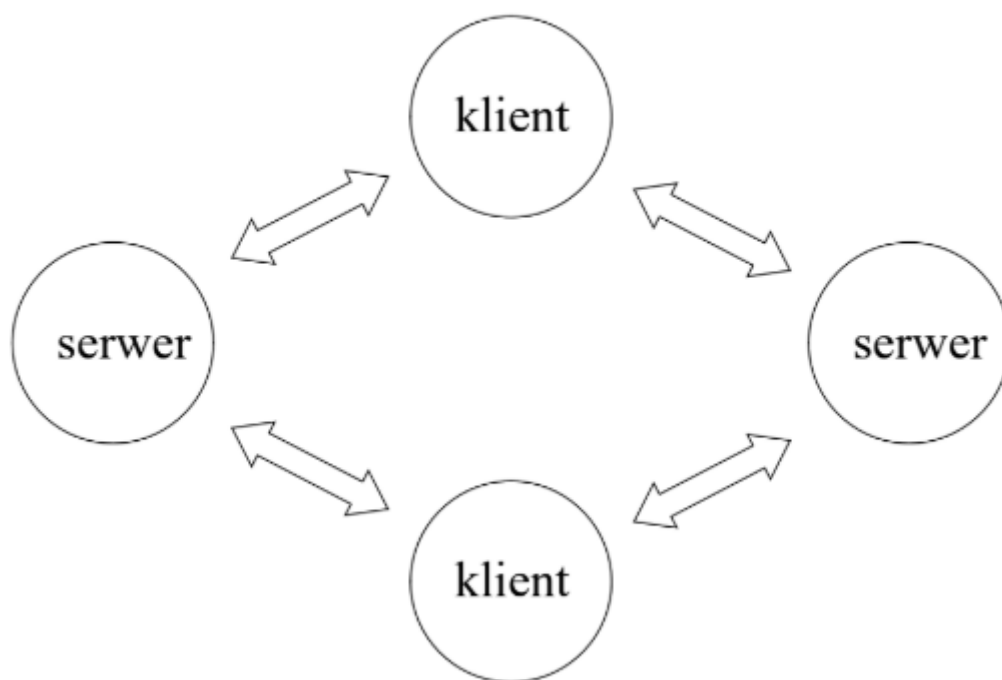
- przekazywanie przez wartość (ang. call-by-value)
 - problem różnic w reprezentacji danych, np. w kodowaniu znaków, uporządkowaniu bajtów, w formatach liczb zmiennopozycyjnych itp.
- przekazywanie przez referencje (ang. call-by-reference)
 - problem zinterpretowania wartości wskaźnika w innej przestrzeni adresowej
- przekazywanie przez kopiowanie i odtwarzanie
 - problem przetaczania (kwestia opisu struktur danych w celu prawidłowego zidentyfikowania wszystkich składowych)

Konwersja danych (reprezentacja kanoniczna)



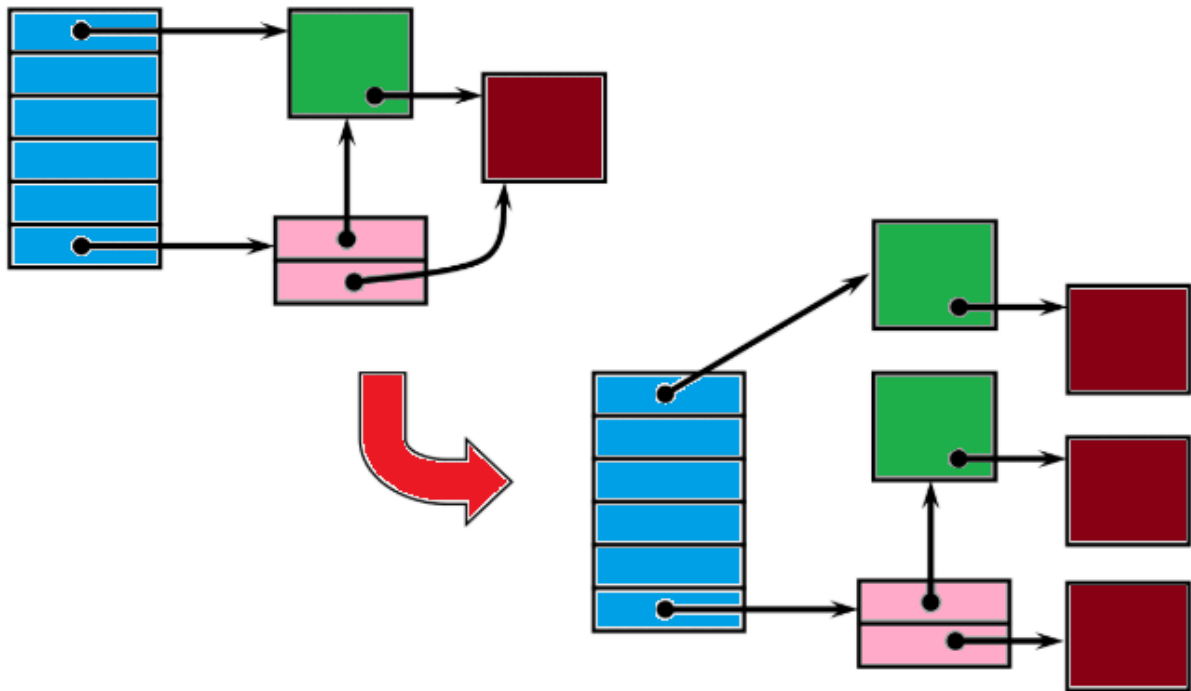
Jak mamy ileś różnych aplikacji działających w innych architekturach i trzeba by dokonywać konwersji np. liczb zmiennopozycyjnych – to przy formacie kanonicznym (przy możliwej utracie dokładności) – każdy klient z niego korzysta, każdy się dostosowuje do tego formatu, no i serwer musi z tego formatu dokonać konwersji do swojej architektury. (I podobnie dane idą od serwera do klientów)

Konwersja bezpośrednia



Można również robić konwersję bezpośrednią, tylko wtedy nam rośnie liczba konwerterów – potencjalnie każdy klient do każdego serwera może coś wysłać (więc tych konwerterów potrzebujemy więcej, chociaż sama konwersja może być najmniej kosztowna)

Problem przetaczania – przykład złożonej struktury danych



Wyobraźmy sobie, że mamy dziwną strukturę (lewy górny rysunek) – mamy tablicę wskaźników, w której mamy wskaźniki różnego rodzaju.

- Mamy wskaźnik na obiekt “zielony”, elementem obiektu zielonego jest wskaźnik na obiekt bordowy.
- Mamy wskaźnik na tablicę wskaźników (różowy) dwuelementową, z których jeden ze wskaźników wskazuje na obiekt zielony, a drugi na obiekt bordowy

Pytanie co się stanie przy typowej konwersji (serializacji i deserializacji)?

Podejście XDR spowoduje, że po serializacji i deserializacji uzyskamy wynik widoczny na prawym dolnym rysunku

Gwarancja wykonania (semantyka błędu) – ważne

- Semantyka co najmniej raz — po uzyskaniu odpowiedzi (wyniku) od serwera klient ma pewność, że wywoływana procedura wykonała się co najmniej raz.

- Semantyka co najwyżej raz — po uzyskaniu odpowiedzi (wyniku) od serwera klient wie, że wywoływana procedura wykonała się dokładnie raz.

W obu powyższych przypadkach mamy sytuację niepewną – gdyż protokół może zwrócić wyjątek. (błąd: nie udało się)

Niektórzy mówią, że “co najmniej raz”, gdyż w tym przypadku nie odpuszczamy – ale to oznacza, że w przypadku błędu mamy zatrzymanie stanu przetwarzania.

A jeżeli akceptujemy błąd? To w przypadku pierwszym nie wiemy nic, może się udało raz, dwa razy, trzy, pięć, dziesięć, pięćdziesiąt – ale tego nie wiemy, po stronie serwerowej mogłoby się udać wielokrotnie.

W przypadku co najwyżej raz gwarantuje nam, że nazwa dzieje się co najwyżej raz – w przypadku niepowodzenia, może się nie udało... albo się udało ale RAZ. Czyli więcej niż raz nie wykonała się dana procedura (ważne w przypadku procedur nie idempotentnych)

Jak należy zinterpretować przypadek wystąpienia błędu (wyjątku) w wywołaniu procedury zdalnej?

- Semantyka dokładnie raz — niemożliwa do uzyskania, jeśli system narażony jest na awarie (np. serwera lub łączy).
- Semantyka ewentualnie — brak gwarancji, procedura mogła się wykonać lub mogła się nie wykonać.

W przypadku sukcesu

- Co najmniej raz: $<1, \text{inf})$
- Co najwyżej raz: raz
- Dokładnie raz: raz
- Ewentualnie: raz

W przypadku błędu:

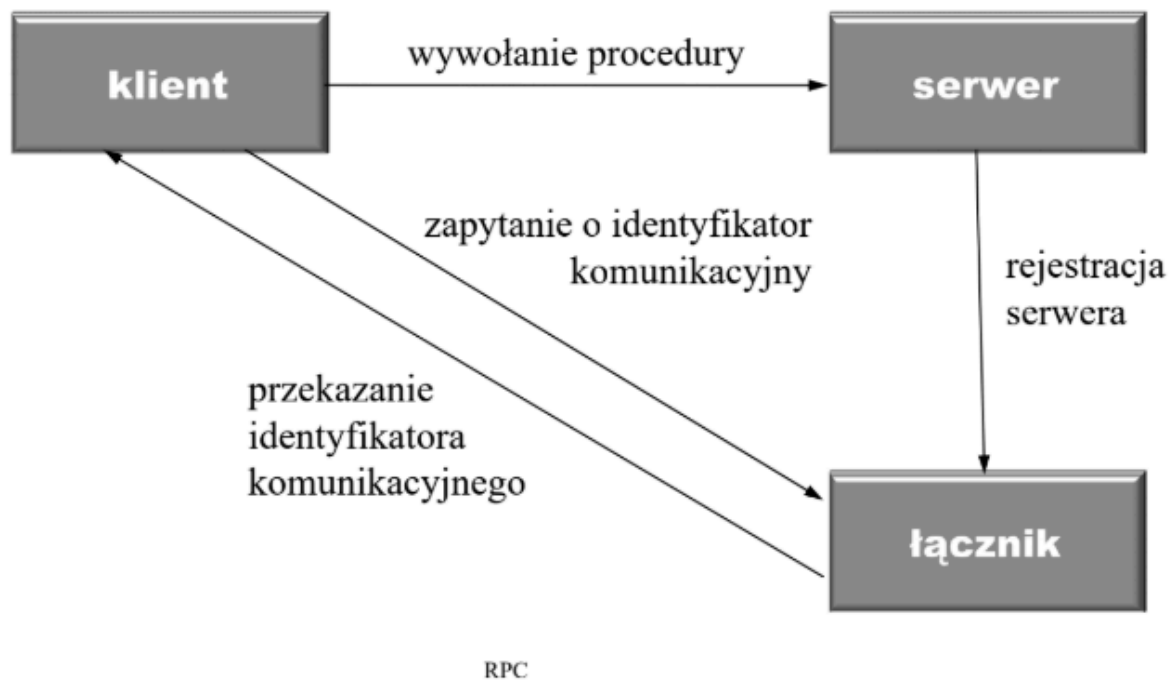
- Co najmniej raz: $<0, \text{inf})$
- Co najwyżej raz: $<0, 1>$

- Dokładnie raz: wcale
- Ewentualnie: wcale

Zagadnienia realizacyjne

- Przetwarzanie interfejsu
 - Generowanie namiastki klienta
 - Generowanie namiastki serwera
 - Generowanie przykładowego programu klienta (client sample)
 - Generowanie wzorca do implementacji procedur zdalnych (template)
 - Generowanie plików do zarządzania kompilacją
- Wiązanie klienta z serwerem
 - Wiązanie statyczne — klient ma wprowadzony na stałe identyfikator komunikacyjny serwera (np. para: adres IP, nr portu).
 - Wiązanie dynamiczne — klient uzyskuje adres serwera za pośrednictwem łącznika (np. portmap lub rpcbind w Sun RPC).
- Obsługa komunikacji klient-serwer
- Realizacja semantyki błędu
- Problem osieroconych obliczeń

Wiązanie dynamiczne klienta z serwerem

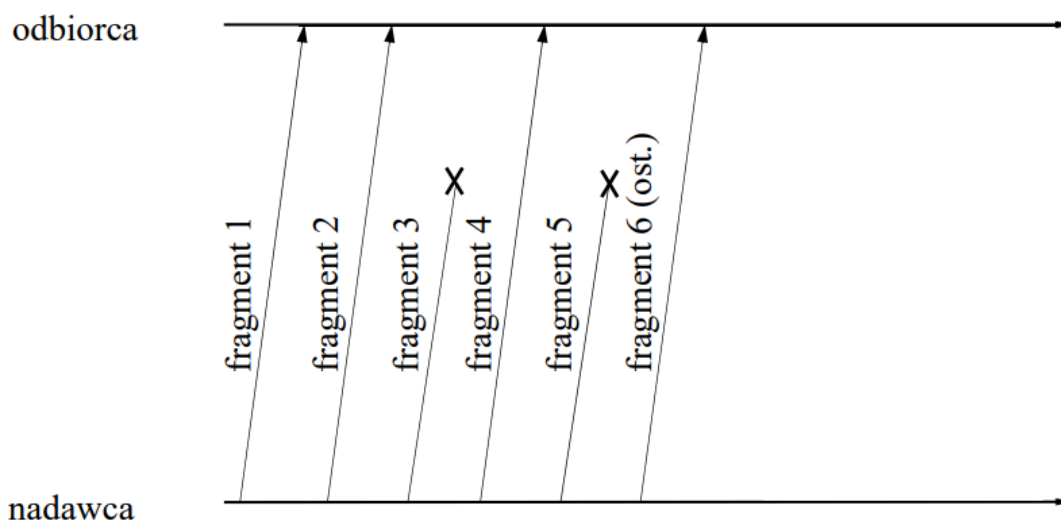


Protokoły warstwy RPC

Protokół BLAST

To sposób przesyłania dużych komunikatów poprzez posegmentowanie na mniejsze komunikaty i przesłanie ich do odbiorcy, który następnie składa te mniejsze komunikaty w całość.

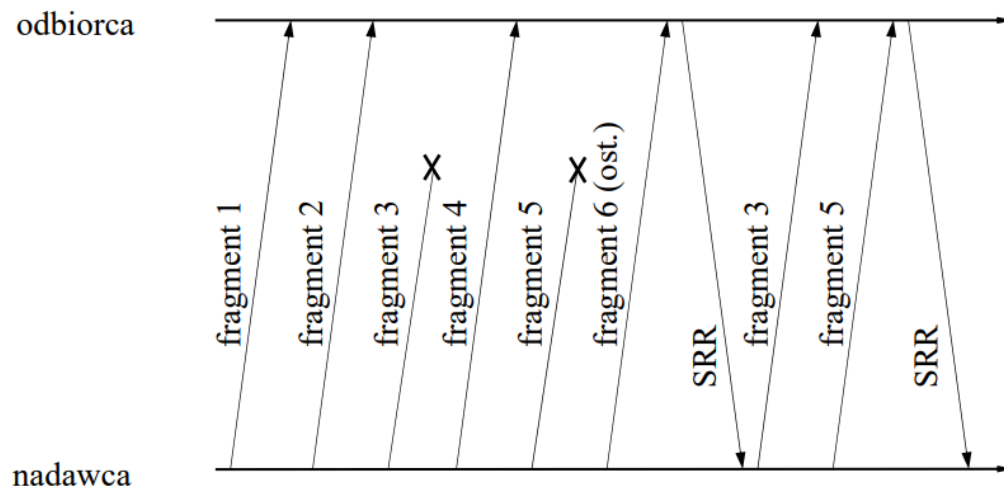
Ostatni komunikat jest specjalnie oznaczony.



W przypadku brakujących elementów:

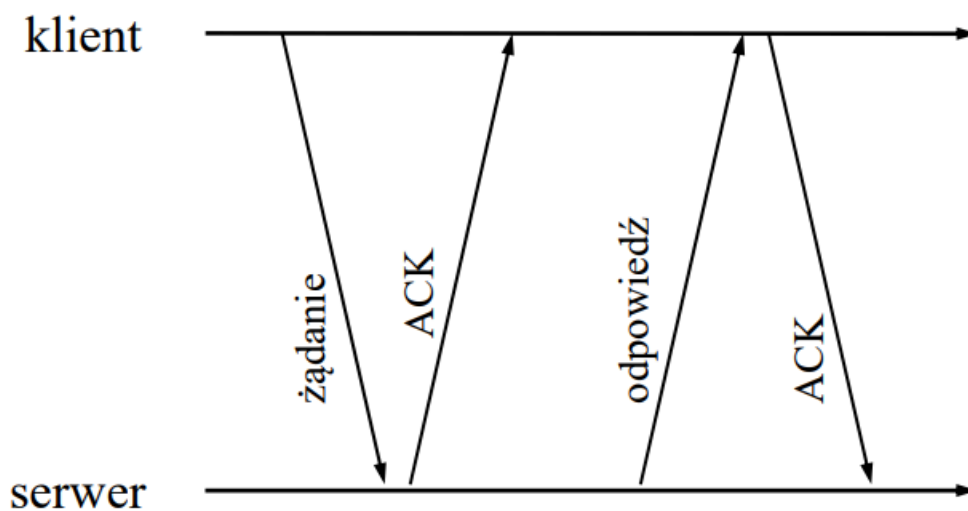
- jeżeli ostatni fragment dotarł, ale wiadomość nie jest cała odbiorca wysyła komunikat SRR, w którym przesyła numery brakujących fragmentów;
- jeżeli ostatni fragment nie dotarł, to odbiorca wysyła wiadomość SRR z brakującymi fragmentami (również zostaje włączone odmierzenie czasu na timerze "RETRY");
- jeżeli odliczanie na timerze "RETRY" dobiegło końca, wiadomość SRR z brakującymi fragmentami zostaje ponownie wysłana (tylko za pierwszym i drugim razem);
- jeżeli odliczanie na timerze "RETRY" dobiegło końca za trzecim razem, odbiorca uwalnia fragmenty (pozbywa się ich z pamięci)

W przypadku pozytywnego odebrania fragmentów po wysłaniu komunikatu SRR, odsyłamy komunikat SRR informujący o odebraniu zaległych fragmentów.



Protokół CHAN

I. komunikacja normalna (zbyt wiele komunikatów)



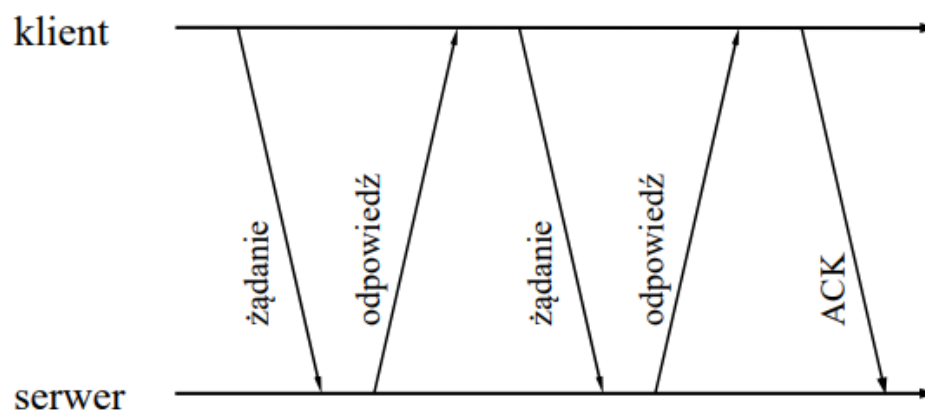
1. Klient wysyła żądanie do serwera,

2. Serwer potwierdza odebranie żądania oraz przystępuje do realizacji żądania
3. Wysyła odpowiedź po jakimś czasie
4. Klient odsyła potwierdzenie odebrania wyniku

II. Domniemanie potwierdzenia

Taki model jest dobry w przypadku gdy czas potrzebny na zrealizowanie żądania, jest krótki, więc serwer może pozbyć się komunikatu potwierdzającego odebranie żądania.

Klient w takim “modelu” powinien założyć, że dana odpowiedź przyjdzie w określonym czasie. Klient kończy komunikację z serwerem za pomocą komunikatu ACK.

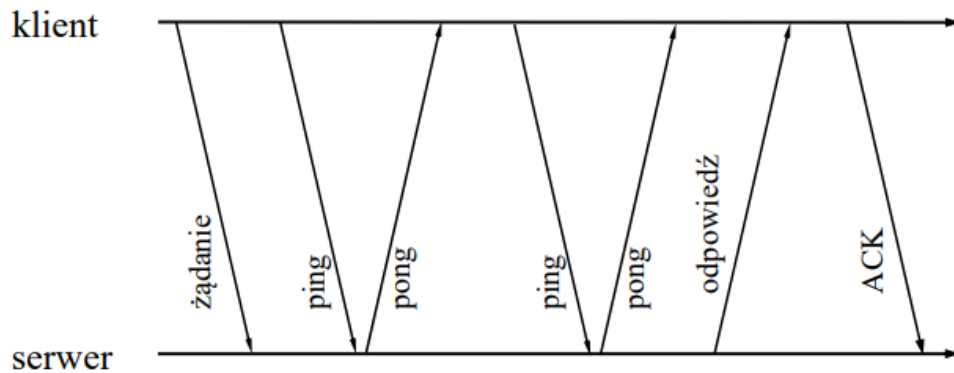


III. Próbkowanie serwera

Osierocenie obliczeń – polega na przykład na tym, że poszło żądanie, a serwer zaprzestał działania (lub nastąpiło partycjonowanie sieci)

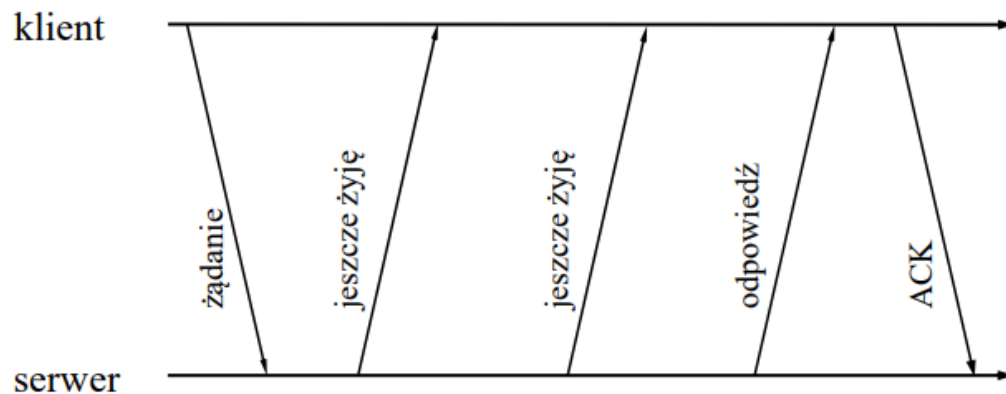
^ W tym momencie klient nie wie czy serwer jeszcze działa, a przy braku odliczania TIMEOUTU oraz różnych metod sprawdzających czy serwer jeszcze istnieje, klient może czekać w nieskończoność.

W takim przypadku dobrym pomysłem jest próbkowanie, czy serwer żyje: przykładowo komunikatami PING-PONG. Jeżeli jeszcze żyje, to odeśle komunikat PONG.



IV. “bicie serca”

Bicie serca to jest sytuacja odwrotna, klient wysyła żądanie – a serwer podejrzewając timeout po stronie klienta i jego “niepokój” będzie co jakiś czas go uspokajał, że żyje “jeszcze żyję” aż w końcu prześle właściwą odpowiedź.



III oraz IV podejście pozwala na weryfikację czy nie nastąpiło osierocenie obliczeń z jednej bądź z drugiej strony (bo klient też może przestać istnieć, podczas gdy serwer działa!)

Realizacja semantyki błędu

- nie można zlokalizować serwera — zgłoszenie wyjątku (nic nie można zrobić, można jedynie zgłosić wyjątek po stronie klienta)
- zaginione żądanie — retransmisja żądania po upływie ustalonego czasu oczekiwania (jeżeli klient zorientuje się, że jest zaginione żądanie, to klient je zretransmituje)
- zaginiona odpowiedź — retransmisja żądania (z perspektywy klienta, zaginiona odpowiedź oraz zaginione żądanie jest takie same)

CO W TAKIM PRZYPADKU ROBI SERWER? – w przypadku procedur nie idempotentnych serwer musi mieć możliwość identyfikacji ponownych żądań

- stosowanie procedur idempotentnych (może ponowić wywołanie jakiegoś zadania, jeżeli nie ma skutków ubocznych)
- numerowanie żądań i retransmisja odpowiedzi (serwer powinien logować odpowiedzi na ewentualną potrzebę retransmisji, aczkolwiek jest to problem w przypadku gdy klient osieroci obliczenia)
- awaria serwera (wg Darka to jest najgorsze)
 - przed podjęciem realizacji → retransmisja żądania (problemu nie ma, wtedy gdy serwer zostanie zrestartowany, gdyż klient ponownie zretransmituje żądanie)
 - po wykonaniu / bądź w trakcie → zgłoszenie wyjątku (z punktu widzenia RPC ma nastąpić zgłoszenie wyjątku, ale nie wiadomo co i kto miałby dalej z tym coś zrobić... wg Darka powinno być odtworzenie stanu żądania)
- awaria klienta — osierocenie obliczeń

Usuwanie osieroconych obliczeń (sposoby)

Eksterminacja

Rejestrowanie działań podejmowanych przez klienta na nośniku niewrażliwym na awarie i usuwanie na tej podstawie osieroconych obliczeń po restarcie klienta (usuwanie tych wykonywanych procedur lub usuwanie również tych przechowywanych odpowiedzi, które nie dotarły do klienta dopóki ich nie potwierdzi)

Reinkarnacja

Każdy restart klienta rozpoczyna nową epokę (identyfikowana przez numer kolejny), po której usuwane są wszystkie obliczenia związane z poprzednią epoką. (Numery epok muszą być przechowywane na niewrażliwym na awarie nośniku)

Łagodna reinkarnacja (mniej istotne)

Reinkarnacja, w której usuwa się tylko te obliczenia rozpoczęte w starej epoce, dla których nie ma właściciela

Wygaśnięcie (Zwrócił na to szczególną uwagę, gdyż jest to rozwiązanie idealne dla protokołu próbkowania)

Przydział określonego czasu T serwerowi na wykonanie procedury. Jeśli wykonanie nie zakończy się w czasie T , serwer musi uzyskać kolejny, przydział pod warunkiem, że obliczenia nie zostały osierocone. Jeśli klient odczeka czas T przy restarcie, osierocone obliczenia same się zakończą.

Protokół SELECT

Po stronie klienta: odwzorowywanie wywołanej procedury na jej identyfikator, przekazywany do serwera

Po stronie serwera: zlokalizowanie wywołanej procedury na podstawie identyfikatora

W praktyce wygląda tak, że protokół RPC osadzamy na pewnych protokołach warstwy transportowej, więc elementy tych warstw rozwiązują część problemów o których mowa jest na tym wykładzie. Te problemy pojawiają się jakbyśmy chcieli zaprojektować protokół RPC od samego zera (bardzo niskiej warstwy).

Wiele problemów rozwiązuje protokół warstwy transportowej, bardzo często rozwiązania typu RPC są nieco odmienne od modelu zaprezentowanym na wykładzie.

Sun RPC/XDR (RPCGEN)

RPCGEN – narzędzie, które pozwala nam na utworzenie znacznej części kodu, na podstawie interfejsu

Właściwości SUN RPC:

- trójwymiarowa identyfikacja procedur (nr programu, nr wersji, nr procedury)
- protokół RPC oparty jest na protokołach warstwy transportowej stosu TCP/IP
- wiązanie dynamiczne (portmap lub rpcbind)
- kanoniczny format reprezentacji danych (XDR)
- opis interfejsu w języku C-podobnym i przetwarzanie przez narzędzie rpcgen

Tworzenie aplikacji

1. Przygotowanie pliku z opisem interfejsu w języku RPC (języku narzędzia **rpcgen**), zawierającego:
2. definicją struktur danych dla przekazywanych parametrów,
3. specyfikację procedur zdalnych, obejmującą:
 - numer programu, numer wersji, numer procedury;
 - typy argumentów;
 - typ zwracanej wartości.
4. wygenerowanie przez rpcgen plików do kompilacji w języku C:
 - pliku nagłówkowego z definicjami stałych;
 - plików z namiastką klienta i namiastką serwera
 - plików zawierających szkielet procedur zdalnych i szkielet programu klienta
 - plik do zarządzania kompilacją (Makefile),
 - plik z funkcjami XDR do konwersji zdefiniowanych typów danych
5. uzupełnienie lub dokonanie niezbędnych modyfikacji w zawartości wygenerowanych plików

- uzupełnienie plików zawierających szkielety procedur zdalnych,
 - uzupełnienie/modyfikacja programu klienta,
 - uzupełnienie/modyfikacja pliku Makefile
6. kompilacja programów,
 7. testowanie działania procedur zdalnych
 - ewentualne przygotowanie aplikacji w wersji zwartej (nie rozproszonej) w celu przetestowania działania samych procedur
 8. przygotowanie programu klienta korzystającego z procedur zdalnych w sposób wynikający z potrzeb aplikacji

Specyfikacja procedur zdalnych

```
program NAZWA_PROGRAMU {  
    version NAZWA_WERSJI1 {  
        typ nazwa_proc1(typ_arg)=nr_proc;  
        typ nazwa_proc2(typ_arg)=nr_proc;  
        ⋮  
    }=numer_wersji;  
    version NAZWA_WERSJI2 {  
        typ nazwa_proc1(typ_arg)=nr_proc;  
        typ nazwa_proc2(typ_arg)=nr_proc;  
        ⋮  
    }=numer_wersji;  
}=numer_programu;
```

program, version – słowa kluczowe

Dostępne zakresy numerów programów

Numery są 32 bitowe

0-0x1FFFFFFF – numery standardowe, zdefiniowane (przydzielane) przez firmę Sun.

0x20000000-0x3FFFFFFF – przydzielane przez użytkownika,
0x40000000-0x5FFFFFFF – zarezerwowane dla aplikacji, które generują
numery programów dynamicznie
0x60000000-0xFFFFFFFF – zarezerwowane

Przykładowy interfejs programu (usunięcie procesu)

```
program REMKILL {  
    version V1 {  
        int rkill(int) = 1;  
    } = 1;  
} = 0x20000002;
```

Przykładowy interfejs programu (z wykorzystaniem struktury)

```
struct rkill_params {  
    int pid;  
    int signum;  
};  
  
program REMKILL {  
    version V1 {  
        int rkill(rkill_params) = 1;  
    } = 2;  
} = 0x20000003;
```

Definicje struktur w XDR oraz C

XDR (rpcgen)

- stałe
`const MAX = 512;`
- typy wyliczeniowe
`enum COLOR {
 red = 1,
 green = 2,
 blue = 4
};`

język C

```
#define MAX 512  
  
enum COLOR {  
    red    = 1,  
    green  = 2,  
    blue   = 4  
};  
typedef enum COLOR  
COLOR;
```

XDR (rpcgen)

- stuktury
`struct ST {
 int a;
 int b;
};`

język C

```
struct ST {  
    int a;  
    int b;  
};  
typedef struct ST ST;
```

XDR (rpcgen)

- unie

```
union UN switch (int d)
{
    case 1: int a;
    case 2: char b;
    default: short c;
};
```

język C

```
struct UN {
    int d;
    union {
        int a;
        char b;
        short c;
    } UN_u;
};
typedef struct UN UN;
```

XDR (rpcgen)

- tablice

```
typedef int tabf[10];
typedef int tabv<10>;
```
- podobnie tablice bajtów

```
typedef opaque btbf[10];
typedef opaque btv<10>;
```
- łańcuchy znaków

```
typedef string s10<10>;
typedef string sbo<>;
```

język C

```
typedef int tabf[10];
typedef struct {
    u_int tabv_len;
    int *tabv_val;
} tabv;
```

- typem bazowym w jest wówczas `char`

Standard XDR

- Opis standardu — RFC 1014
- Kanoniczna reprezentacja danych oparta na formacie IEEE
- Deklaratywny język opisu struktur danych (zbliżony do języka C)
- Koncepcja konwersji oparta na
 - potokach — miejscach przechowywania danych w formacie XDR
 - filtrach — procedurach konwersji pomiędzy własnym formatem maszyny a formatem kanonicznym (w obu kierunkach)

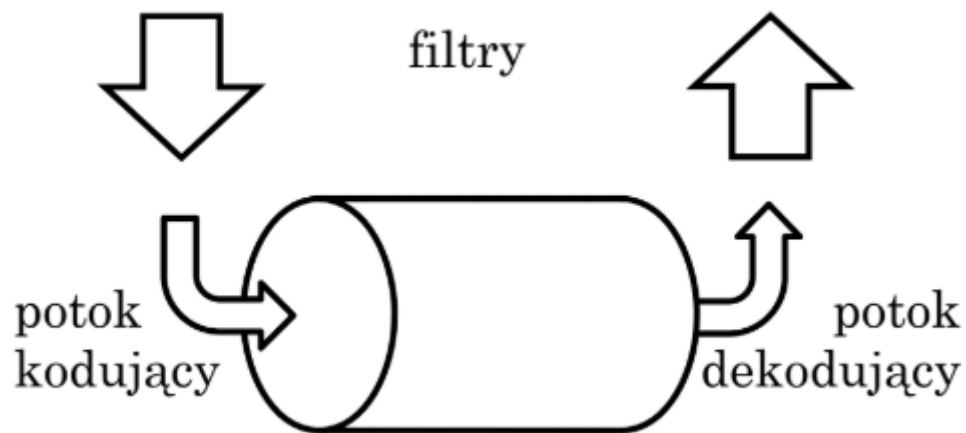
Potok XDR

- Potok jest miejscem składowania danych XDR, a dokładniej środkiem dostępu do nich w celu:
 - zapisu — potok kodujący (encoding)
 - odczytu — potok dekodujący (decoding)
- Rodzaje potoków:
 - potok na standardowym wejściu-wyjściu — na otwartym pliku
 - potok w pamięci — w obszarze pamięci opisanym przez adres i rozmiar w bajtach
 - potok komunikatów (rekordów) — na strumieniu danych, zdefiniowanym wraz z procedurami obsługi tego strumienia (zapis, odczyt)

Filtr XDR

- Filtr jest procedurą konwersji, służącą do realizacji dostępu do danych w potoku w celu zapisu (kodującą) lub odczytu (dekodującą) — zależnie od kierunku działania potoku.
- Rodzaje filtrów
 - filtr prosty — służy do konwersji typów prostych
 - filtr złożony — służy do konwersji typów złożonych (np. tablicowych, wskaźnikowych)
 - filtr pochodny — połączenie innych filtrów w ramach jednej procedury filtrującej

Potoki i filtry XDR



Tu widoczny jest jeden filtr, ale działający w dwóch kierunkach (potok określa kierunek filtru)

Filtry proste

Filtr dla:

- Filtr dla: typów prostych: `bool_t`, `char`, `short`, `int`, `long`, (również bez znaku) `float`, `double`
- typu wyliczeniowego (`enum`)
- typu `void`

Ogólna postać procedury filtrującej (na przykładzie typu `unsigned int`):

- `bool_t`
`xdr_u_int(XDR* xdrs,`
`unsigned int *ptr);`

Filtry złożone

Filtr do konwersji

- tablic bajtów o ustalonej lub zmiennej wielkości — `xdr_opaque`, `xdr_bytes`
- tablic elementów określonego typu o ustalonej lub zmiennej wielkości — `xdr_vector`, `xdr_array`
- łańcuchów znaków — `xdr_string`, `xdr_wrapstring`

- typów wskaźnikowych — xdr_reference, xdr_pointer
- unii — xdr_union (w praktyce dla statycznie zdefiniowanych typów unijnych stosowany jest filtr pochodny)

Ogólna postać filtra złożonego

```
xdr_typ(XDR* xdrs, ptr, ...,
        [xdrproc_t elproc]);
```

- W przypadku typu statycznego (ustalona zajętość pamięci) wskaźnik ptr, ma postać char *ptr, w przypadku typu o zmiennej wielkości (konieczność dynamicznej alokacji pamięci) char **ptr.
- Jeśli typ podstawowy (w przypadku tablic, wskaźników) nie jest z góry określony, konieczne jest wskazanie filtra XDR do konwersji elementu typu podstawowego.

Filtry złożone do konwersji tablic (aczkolwiek raczej niepotrzebne)

```
xdr_array(xdrs, arrp, sizep, maxsize, elsize, elproc)
    XDR *xdrs;
    char **arrp;
    u_int *sizep, maxsize, elsize;
    xdrproc_t elproc;
xdr_vector(xdrs, arrp, size, elsize, elproc)
    char *arrp;
    u_int size, elsize; // pozostałe parametry – j.w.
xdr_bytes(xdrs, sp, sizep, maxsize)
    char **sp; // pozostałe parametry – j.w.
xdr_opaque(xdrs, cp, cnt)
    char *cp;
    u_int cnt;
```

Filtry złożone do konwersji łańcuchów znaków

```
xdr_string(xdrs, sp, maxsize)
    XDR *xdrs;
    char **sp;
    u_int maxsize;
xdr_wrapstring(xdrs, sp)
    ≡ xdr_string(xdrs, sp, MAXUN.UNSIGNED );
```

`xdr_wrapstring` nie wymaga podania rozmiaru (liczby znaków łańcucha)

Filtry złożone do konwersji struktur wskaźnikowych

```
xdr_reference(xdrs, pp, size, proc)
    XDR *xdrs;
    char **pp;
    u_int size;
    xdrproc_t proc;
xdr_pointer(xdrs, objpp, objsize, xdrobj)
    XDR *xdrs;
    char **objpp;
    u_int objsize;
    xdrproc_t xdrobj;
```

`xdr_pointer` w przeciwieństwie do `xdr_reference` interpretuje wskaźnik pusty, co umożliwia obsługę rekurencyjnych struktur danych

Przykład filtru pochodnego

Definicja struktury

```
struct struktura {  
    int x;  
    long y;  
    char c;  
    short s;  
};
```

Filtr XDR

```
bool_t  
xdr_struktura (XDR *xdrs,  
               struktura *objp) {  
    if (!xdr_int (xdrs,  
                  &objp->x)) return FALSE;  
    if (!xdr_long (xdrs,  
                   &objp->y)) return FALSE;  
    if (!xdr_char (xdrs,  
                   &objp->c)) return FALSE;  
    if (!xdr_short (xdrs,  
                    &objp->s)) return FALSE;  
    return TRUE;  
}
```

Zarządzanie pamięcią

- Przekazanie adresu pustego wskaźnika (typu char**) przy konwersji dekodującej spowoduje dynamiczną alokację pamięci przez filtr XDR.
- Zwolnienie obszaru dynamicznie zaalokowanej pamięci przez filtr XDR musi nastąpić w programie aplikacyjnym.
- Ogólna postać funkcji zwalniania pamięci:
xdr_free(xdrproc_t proc,
 char* objp);

Tworzenie potoków

- Potok na standardowym wejściu-wyjściu:
void xdrstdio_create(XDR *xdrs,
 FILE *file, enum xdr_op op);
- Potok w pamięci:
void xdrmem_create(XDR *xdrs, char *addr,
 u_int size, enum xdr_op op);

Potok komunikatów:

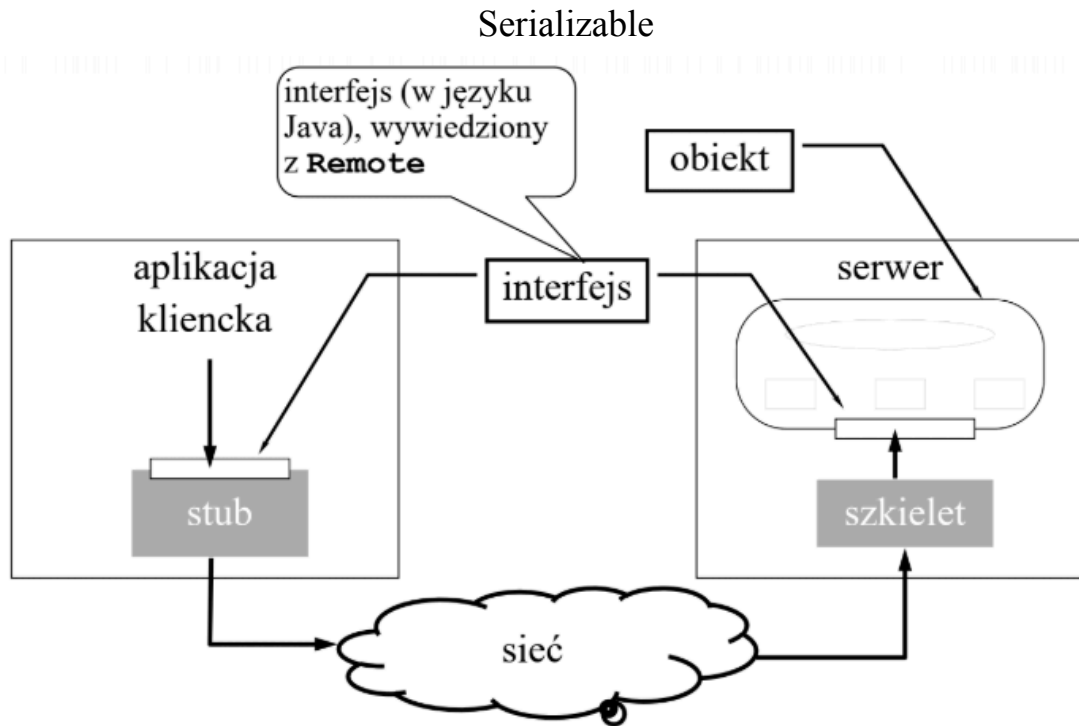
- `void xdrrec_create(XDR *xdrs,
 u_int sendsize, u_int recvsize,
 char *handle,
 int (*readit)(char*,char*,int),
 int (*writeit)(char*,char*,int));`

Potoki komunikatów

- Ustalenie kierunku potoku musi nastąpić po jego utworzeniu (np. `xdrs -> x_op = XDR_ENCODE`)
- Rozmiary buforów (parametry `sendsize` i `recvsize`) mogą mieć wartość 0, co oznacza przyjęcie wartości domyślnych.
- Gdy konieczne jest opróżnienie bufora wyjściowego (wysłanie danych) lub zapełnienie bufora wejściowego, wywoływana jest odpowiednia funkcja (`writeit`, `readit`) z trzema parametrami:
 - uchwycem `handle`
 - adresem bufora
 - liczbą bajtów do zapisu/odczytu
- Oznaczenie końca rekordu (przy zapisie)
`xdrrec_endofrecord(XDR *xdrs,
 int sendnow);`
- Pomińnięcie reszty rekordu (przy odczycie)
`xdrrec_skiprecord(XDR *xdrs);`
- Sprawdzenie zakończenia strumienia (przy odczycie)
`xdrrec_eof(XDR *xdrs);`

Wywołanie metod zdalnych

Model systemu



- Interfejs może być dostarczony w języku implementacji w przeciwieństwie do RPC

Obiekt zdalny w środowisku Java

- Mechanizm RMI umożliwia tworzenie obiektów zdalnych (brak bezpośredniego wsparcie dla tworzenia obiektów rozproszonych)
- Jedyna forma zdalnego dostępu polega na wywoływaniu metod wyspecyfikowanych w interfejsie wywiedzionym (dziedziczącym) `java.rmi.Remote`
- Interfejs zdefiniowany jest w języku implementacji
- Obiekt może implementować wiele interfejsów
- Ten sam interfejs może być implementowany w wielu klasach i występować w wielu instancjach (obiektych) każdej klasy
- Interfejs traktowany jest jak typ danych

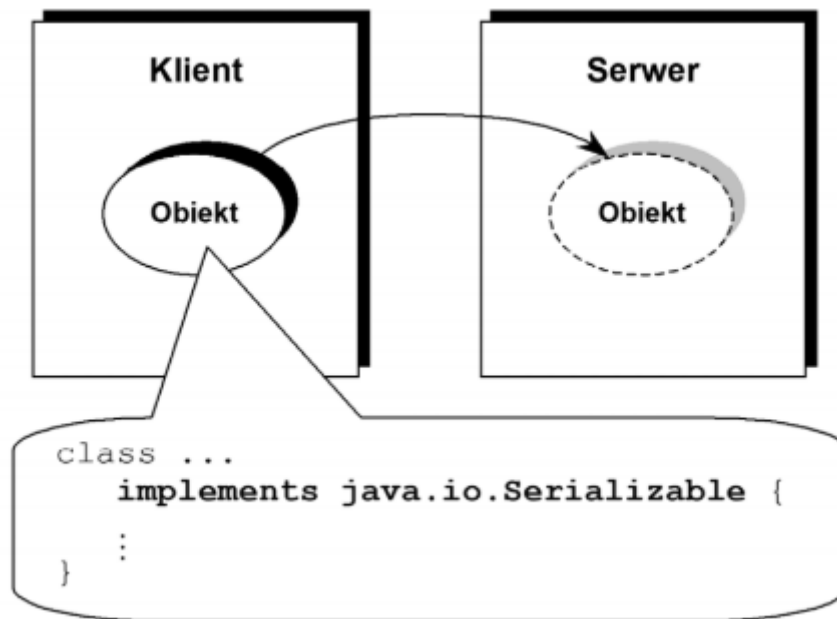
Dostępność obiektu

- Informacja o typie obiektu (czyli o zdalnym interfejsie) dostępna jest w czasie kompilacji.
- Wiązanie obiektu jest jawne i odbywa się w czasie wykonania
- Trwałość obiektu
 - obiekt udostępniany przez `UnicastRemoteObject` ma charakter przejściowy (istnieje tylko w czasie działania serwera)
 - dostępny jest mechanizm obiektów aktywowalnych, ale brak bezpośredniego wsparcia dla utrwalania stanu obiektu

Przekazywanie obiektów jako parametrów

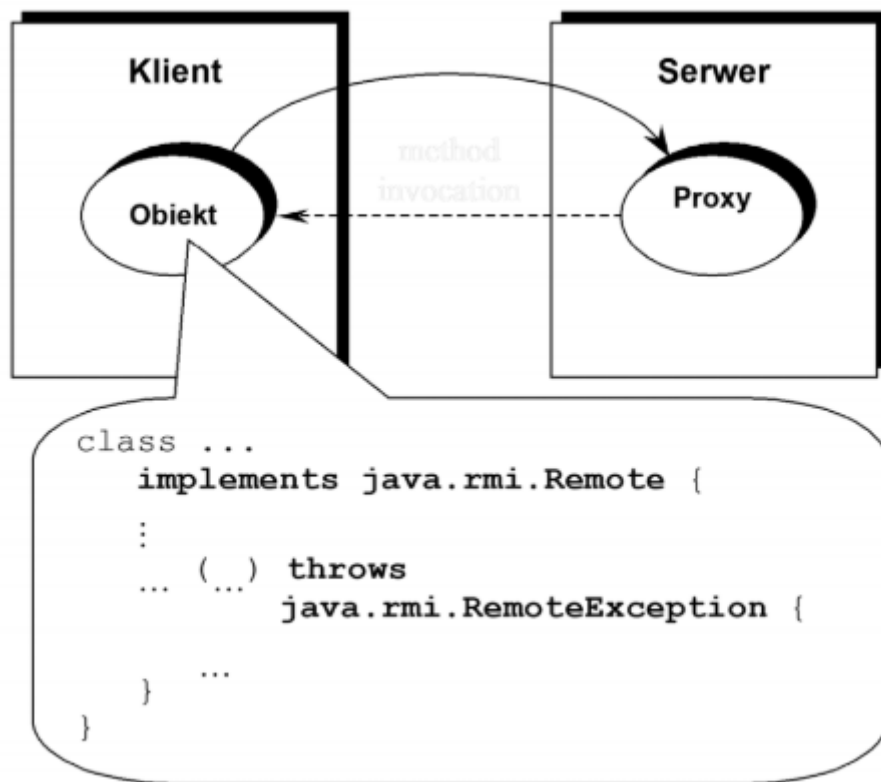
- Obiekty mogą być przekazywane przez wartość (kopię) — konieczna jest deklaracja implementacji interfejsu `java.io.Serializable` w klasie obiektu.
- Obiekty zdalne (implementujące interfejs `Remote`) przekazywane są przez referencję — do zdalnej metody przekazywana jest zdalna referencja, za którą udostępniany jest proxy (stub).
- Brak bezpośredniego wsparcia dla przekazywania parametrów przez kopiowanie i odtwarzanie.

Przekazywanie przez wartość – przykład w Java RMI



Klient wysyła kopię obiektu, do jakiegoś z jego zdalnych obiektów – klasa tego obiektu implementuje `java.io.Serializable`

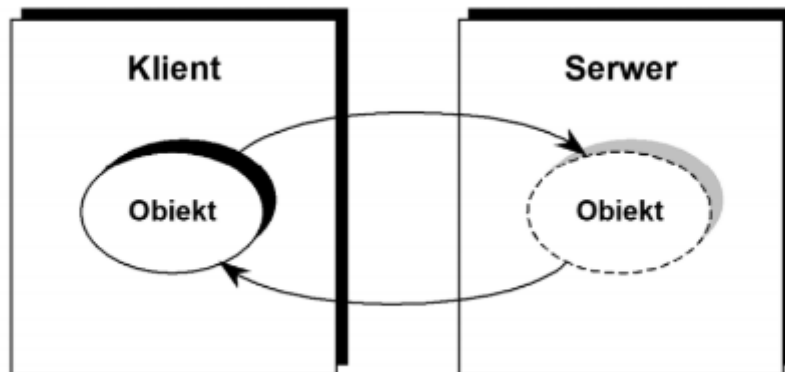
Przekazywanie przez referencję



Jeśli mamy przekazywanie przez referencję, to jest jakaś klasa, która implementuje klasę pochodną, która implementuje interfejs `java.rmi.Remote` – funkcje wyrzucają wyjątek `java.rmi.RemoteException`

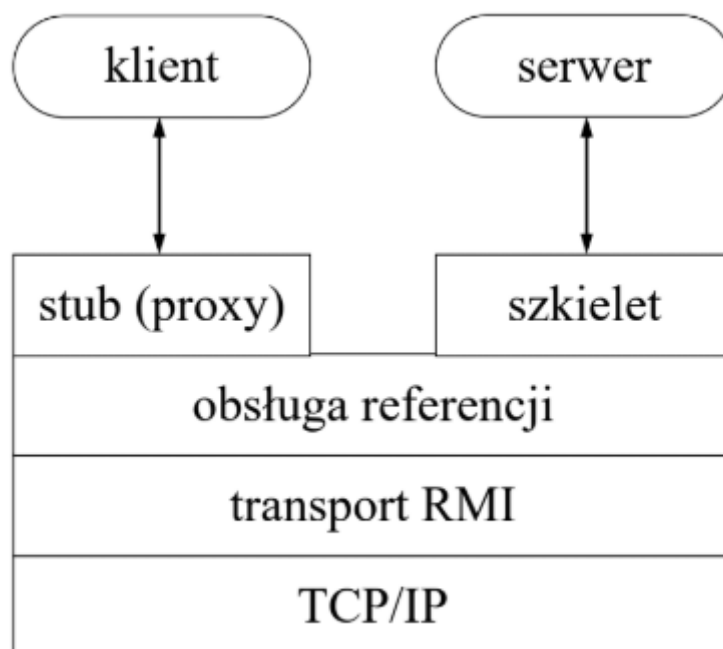
Zakładając, że ta klasa nie jest pusta (czyli wywołujemy jakąś zdalną metodę któregoś obiektu na serwerze i przekazujemy mu jako parametr obiekt klasy `java.rmi.remote`) to w rzeczywistości on dostanie stub i za pomocą tego stuba będzie mógł wywoływać operacje, które będą realizowane po stronie klienta.

Przekazywanie przez kopiowanie i odtwarzanie



Przekazujemy obiekt jako parametr, dostajemy z punktu widzenia Java inny obiekt z odpowiedzią, następnie wartości obiektu u Klienta zostaną zastąpione wartościami otrzymanymi przez serwer.

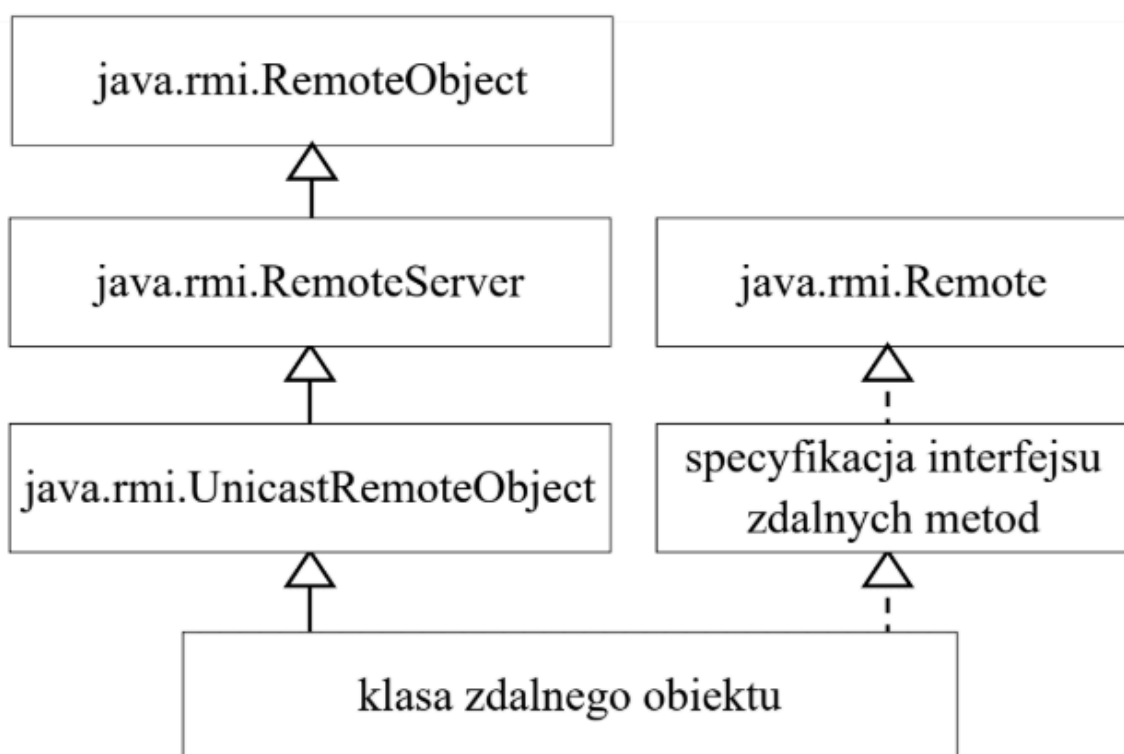
Architektura RMI



RMI bazuje na protokole TCP i tutaj według Dariusza uznania, nie jest w sposób to zmienić. Normalnie bazuje się na protokole TCP, transporcie RMI i obsłudze referencji.

To co dotyczy nas jako programistów to klient i serwer udostępniający obiekty. Ewolucja RMI jest strasznie sympatyczna (Darek kiedyś chciał go wyrzucić z programu nauczania, ale ostatnie lata przekonały go, że bardzo łatwo jest uzyskać szybki efekt za pomocą Javy RMI).

[Hierarchia klas w definicji zdalnego obiektu](#)



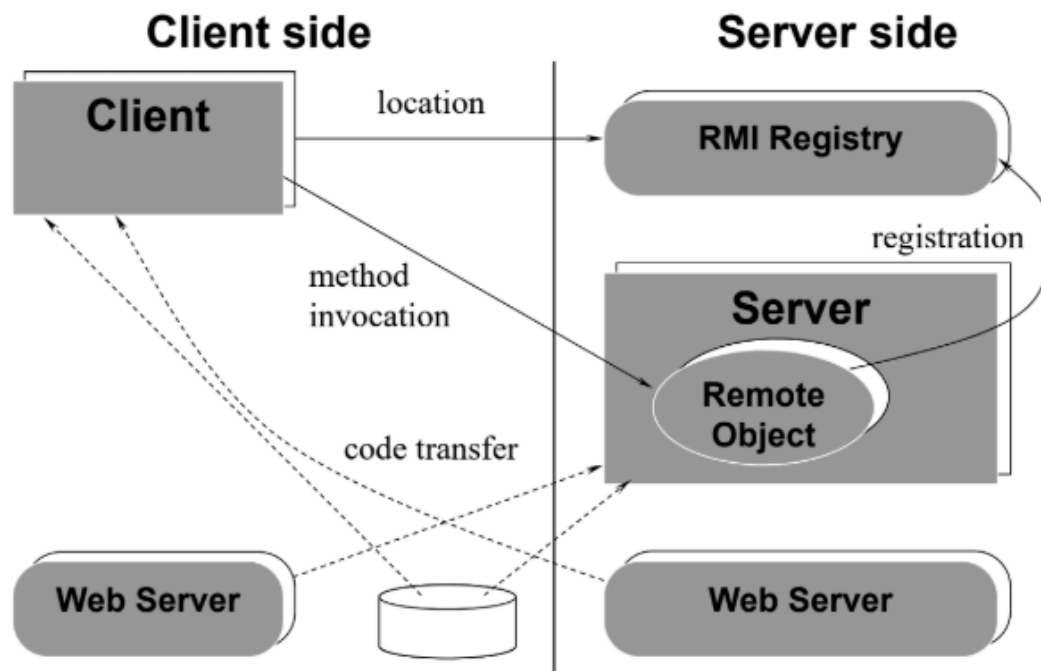
Tworzenie aplikacji rozproszonej w środowisku java RMI

1. Zdefiniowanie i implementacja odpowiednich klas (w szczególności klas dla obiektów dostępnych zdalnie)
 - zdefiniowanie interfejsu pochodnego od `Remote`
 - zdefiniowanie klasy wywiedzionej z klasy `java.rmi.server.UnicastRemoteObject`, implementującej interfejs pochodny od `Remote` lub użycie statycznej metody `exportObject` klasy `UnicastRemoteObject`
2. Kompilacja źródeł (`javac`, `rmic`)
 - `javac xxx.java ⇔ xxx.class`
 - `rmic xxx ⇔ xxx_Stub.class`
`xxx_Skel.class`

zbędne od Java 5

we wcześniejszych wersjach Javy
3. Udostępnienie wygenerowanego kodu klas
 - wspólny system plików
 - kopia kodów klas w różnych systemach plików
 - udostępnianie kodu przez serwer www
4. Uruchomienie aplikacji
 - uruchomienie `rmiregistry` (name server)
 - uruchomienie serwera: utworzenie zdalnych obiektów i ich rejestracja w `rmiregistry`
 - uruchomienie klienta: zlokalizowanie zdalnych obiektów (odwołanie do `rmiregistry`) i wywoływanie zdalnych metod

Komunikacja między klientem a serwerem



- Klient
 - Serwer
 - RMI Registry (funkcjonuje poza serwerem, “ma swój osobny serwer”)
1. Serwer tworzy jeden lub więcej zdalnych obiektów, który następnie rejestruje w RMI Registry
 2. Klient chcący uzyskać dostęp do obiektu komunikuje się z rejestrem, otrzymuje odpowiednie informacje
 3. Klient następnie może już wywoływać zdalnie obiekty

[Interfejs java.rmi.Remote](#)

Komunikacja pomiędzy serwerem (obiektem) a klientem jest określona przez definicję interfejsu pochodnego od interfejsu Remote. Klasa zdalnego obiektu musi implementować ten interfejs.

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface Compute extends Remote {
    Object executeTask(Task t) throws RemoteException;
}
```

Zdalne udostępnianie obiektu (serwer)

Definicja klasy obiektu:

```
import java.rmi.*;
import java.rmi.server.*;
import compute.*;

public class ComputeEngine extends UnicastRemoteObject
    implements Compute {

    public ComputeEngine() throws RemoteException {
        super();
    }
    public Object executeTask(Task t) {
        return t.execute();
    }
}
```

Utworzenie i rejestracja zdalnego obiektu (strona serwera)

```
public static void main(String[] args) {
    if (System.getSecurityManager() == null) {
        System.setSecurityManager(new RMISecurityManager());
    }
    String name = "//host:1099/Compute";
    try {
        Compute engine = new ComputeEngine();
        Naming.rebind(name, engine);
        System.out.println("ComputeEngine ok");
    } catch (Exception e) {
        System.err.println("ComputeEngine excep."
            + e.getMessage());

        e.printStackTrace();
    } } }
```

1099 - domyślny portu na którym RMI Registry nasłuchuje

Zdalne wywołanie metody (strona klienta)

```
import java.rmi.*;
public class launchComp {
    public static void main(String args[]) {
        if (System.getSecurityManager() == null){
            System.setSecurityManager(new RMISecurityManager());
        }
        try {
            String name = "/" + args[0] + "/Compute";
            Compute comp = (Compute) Naming.lookup(name);
            CompArea task = new CompArea(...);
            BigDecimal area =
                (BigDecimal) (comp.executeTask(task));
            System.out.println(area);
        } catch (Exception e) {
            ...
        } } }
```

Lokalizowanie zdalnych obiektów

- Rejestr — wiąże z nazwami i udostępnia zdalne obiekty
- Tworzenie rejestru — `rmiregistry [<port number>]`
- Klasa `LocateRegistry` — umożliwia tworzenie rejestru (obiekty klasy `Registry`)
- Klasa `Registry` — klasa obiektu-rejestru
- Klasa `Naming` — ułatwia korzystanie z rejestru, umożliwiając jego specyfikację w adresie URL

ØMQ (ZeroMQ)

- Mechanizm komunikacji ze zintegrowanym kolejkowaniem komunikatów (brak brokera)
- Interfejs wzorowany na gniazdach BSD
- Interfejs dostępny między innymi dla języka C, C++, C#, Java, Python, Ruby, Ada
- Zdefiniowane schematy komunikacji (REQ-REP, PUB-SUB, PUSH-PULL)
- Brak struktury komunikatu (komunikat jest sekwencją bajtów, których typ/struktura zdefiniowana jest na poziomie aplikacji)

Ogólny schemat komunikacji

Proces 1

1. utworzenie kontekstu
2. utworzenie gniazda
3. połączenie gniazda
4. wysłanie komunikatu
5. inne operacje komunikacyjne
6. zamknięcie gniazda
7. usunięcie kontekstu

Proces 2

1. utworzenie kontekstu
2. utworzenie gniazda
3. związanie gniazda
4. odbiór komunikatu
5. inne operacje komunikacyjne
6. zamknięcie gniazda
7. usunięcie kontekstu

Kontekst

- Kontekst jest zbiorem gniazd.
- Tworzenie kontekstu
`void *context = zmq_ctx_new ();`
- Zalecenie: 1 kontekst na proces.

- Usuwanie kontekstu
`zmq_ctx_destroy (context);`
lub
`zmq_ctx_term (context);`
 - zakończenie wszystkich operacji blokujących
 - oczekiwanie na zakończenie wysyłania komunikatów
 - oczekiwanie na zamknięcie wszystkich gniazd

Tworzenie gniazda

- Wywołanie:
`void *zmq_socket (void *context, int type);`
- Podstawowe typy gniazd:
 - REQ i REP (naprzemienne wysyłanie i odbiór)
 - PUB i SUB (publikowanie do wielu subskrybentów)
 - PUSH i PULL (przetwarzanie potokowe)

Wiązanie i łączenie gniazd

- Wiązanie gniazda:
`void * gniazdo = zmq_socket (... , ...);`
`zmq_bind (gniazdo, "tcp://*:5556");`
- Łączenie gniazd:
`void * gniazdo = zmq_socket (... , ...);`
`zmq_connect (gniazdo, "tcp://*:5556");`
- Łączyć można tylko „kompatybilne” gniazda.

JMS

Cechy MOM

- Uniezależnienie funkcjonowania składników aplikacji od dostępności informacji o interfejsach innych składników
- Uniezależnienie funkcjonowania warstwy komunikacyjnej (kanału komunikacyjnego) od działania (obecności) komunikujących się procesów => komunikacja nieustanna
- Mechanizm komunikacji pośredniej
 - oparty na identyfikacji miejsca pośredniczącego (tzw. skrzynki, a nie adresu nadawcy/odbiorcy)
 - komunikujące się strony nie muszą znać się wzajemnie
- Łatwość wdrożenia komunikacji asynchronicznej

Koncepcja komunikacji oparta na kolejkowaniu

- Komunikacja punkt-punkt
- Organizacja warstwy komunikacyjnej w postaci systemu kolejek (ang. queue) {skrzynka kolejka} realizowanych w oparciu o zasoby pamięci, w tym pamięci dyskowej (gwarancja trwałości na wypadek awarii => niezawodność)
- Udostępnienie mechanizmów komunikacji polegających na:
 - umieszczeniu komunikatów w kolejkach,
 - pobieraniu komunikatów z kolejek.

Paradygmat Publish/Subscribe

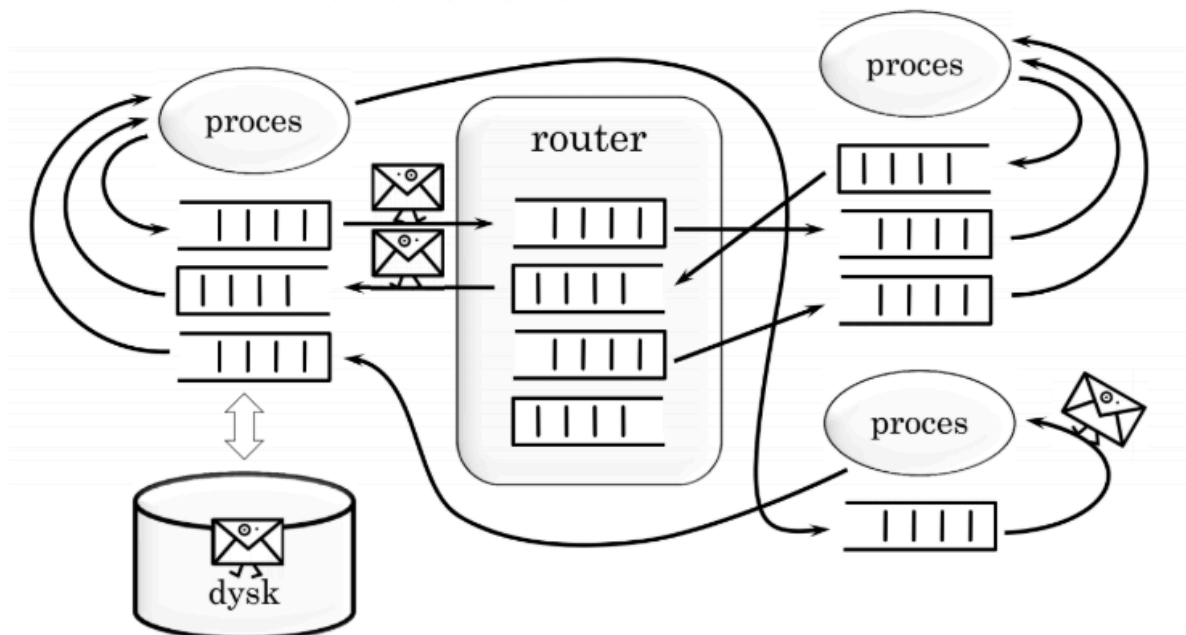
- Komunikacja jeden do wielu (potencjalnie wielu do wielu)
- Strona publikująca (nadawca) udostępnia treść związaną z określonym tematem (ang. topic) {skrzynka $\equiv$ temat}

- Środowisko komunikacyjne (usługa) przekazuje treść udostępnionych wiadomości odbiorcom (subskrybentom), którzy zarejestrowali (zapisali) się wcześniej na dany temat

Model systemu typu MOM – podstawowe pojęcia

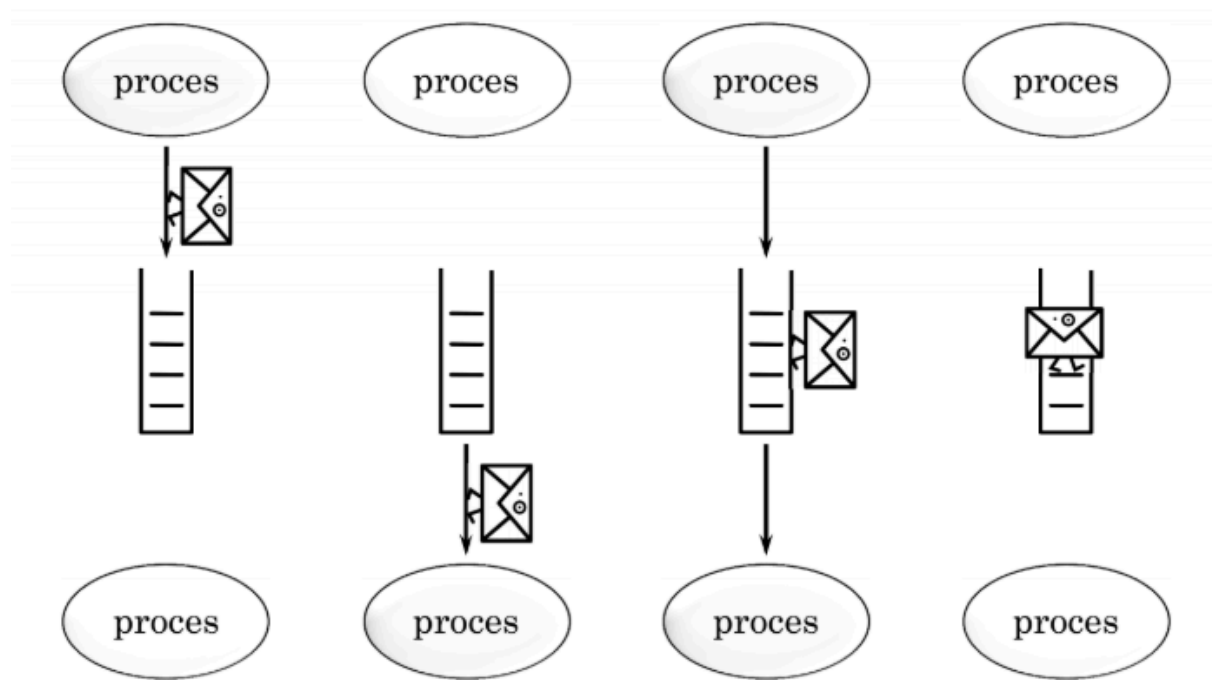
- Wiadomość (ang. message) — porcja danych (często z dodatkowymi własnościami) składowana w kolejce
- Kolejka (ang. queue) — miejsce przechowywania wiadomości (komunikatów) {temat $\equiv$ kolejka dla wielu odbiorców, ang. multiconsumer queue}
- Proces (ang. process) — element aplikacji, zlecający operacje dotyczące wiadomości w kolejce
- Zarządca zbioru kolejek — moduł (proces na określonym węźle, dostawca, broker), odpowiedzialny za wykonywanie operacji na kolejkach, np. tworzenie, usuwanie, lokalizowanie kolejek, definiowanie atrybutów kolejek itp.)

Funkcjonowanie systemu kolejkowania komunikatów



Coś mówił na ten temat?

Warianty komunikacji



- Szare zamalowanie - aktywność procesu
- Białe zamalowanie - brak aktywności
- 1 i 2 obrazek:
 1. Proces górny wysyła komunikat, proces dolny jest nieaktywny
 2. Proces dolny uaktywnia się oraz pobiera komunikat
- 3 i 4 obrazek:
 1. Oba procesy są aktywne, wiadomość została umieszczona w kolejce
 2. Proces drugi przed otrzymaniem wiadomości, został dezaktywowany ("komunikat sobie siedzi i czeka")

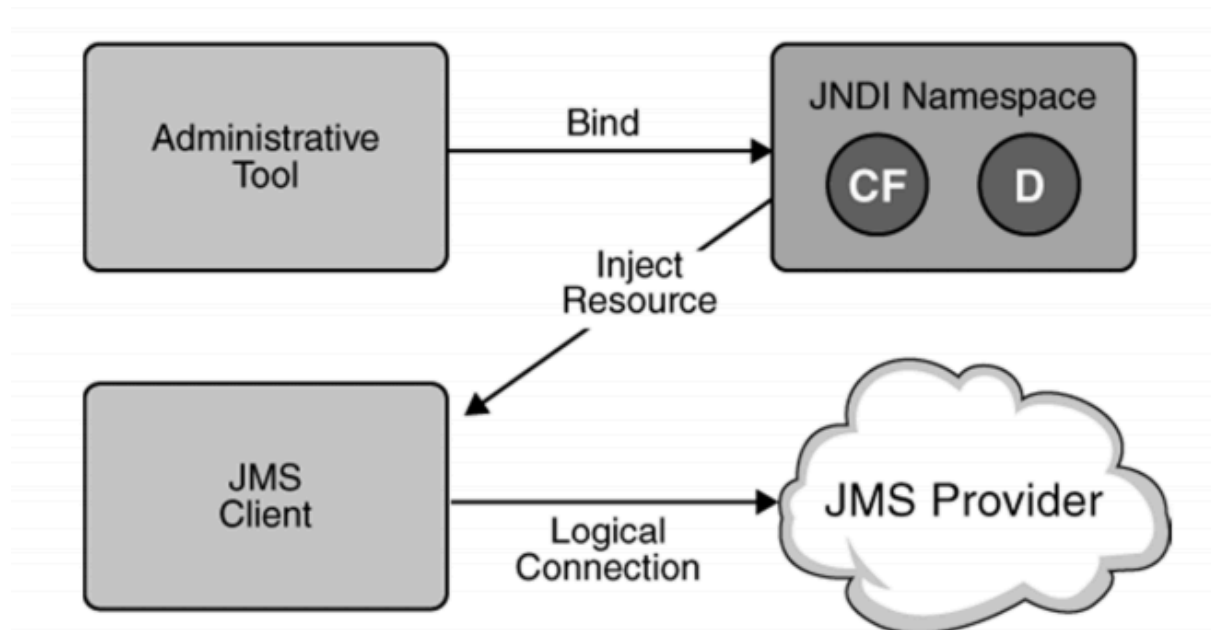
Komponenty JMS

- Dostawca JMS (ang. JMS provider) — implementacja interfejsów JMS, administracja, sterowanie
- Klienci JMS — aplikacje i komponenty wysyłające i odbierające komunikaty
- Wiadomości — obiekty do przenoszenia informacji
- Obiekty zarządzania (ang. administered objects) – prekonfigurowane obiekty na potrzeby zarządzania:
 - cele (ang. destinations)
 - fabryki połączeń (ang. connection factories)

Funkcjonalność JMS

- Nieustanna, niezawodna, asynchroniczna komunikacja międzyprocesowa
- Transakcyjna interakcja z dostawcą JMS
- Modele komunikacji (messaging domains)
 - punkt-punkt (ang. point-to-point)
 - subskrypcji (ang. publish/subscribe)

Architektura JMS

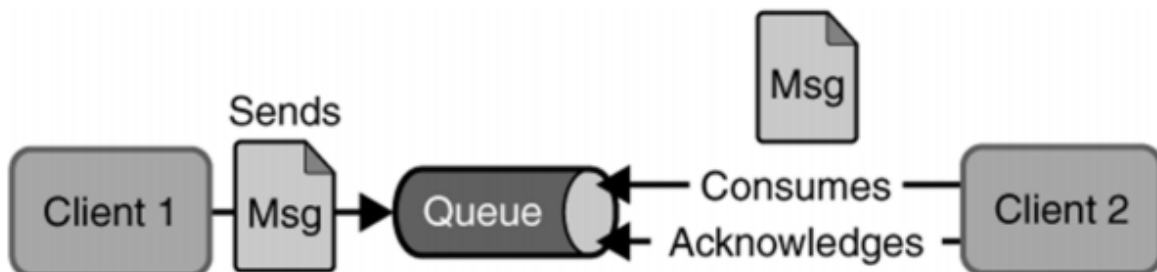


JNDI – Java Naming and Directory Interface, czyli pewnego rodzaju usługa katalogowa w której rejestruje się pewne rzeczy (Connecting factories, destinations itd.)

Można uprościć wystarczy Client i Provider

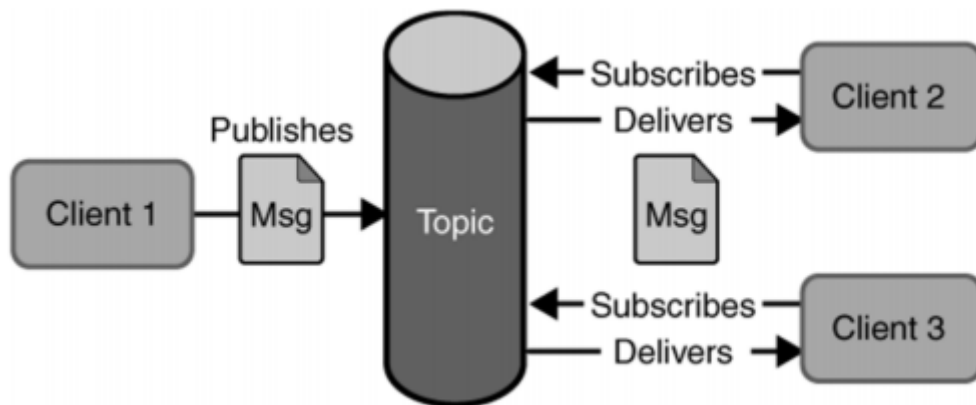
Model punkt-punkt

- Wysyłanie i odbiór poprzez dostęp do kolejki
- Wiadomości pozostają w kolejce do czasu odbioru lub przedawnienia
- Każda wiadomość ma 1 konsumenta
- Brak ograniczeń czasowych



Model subskrypcji

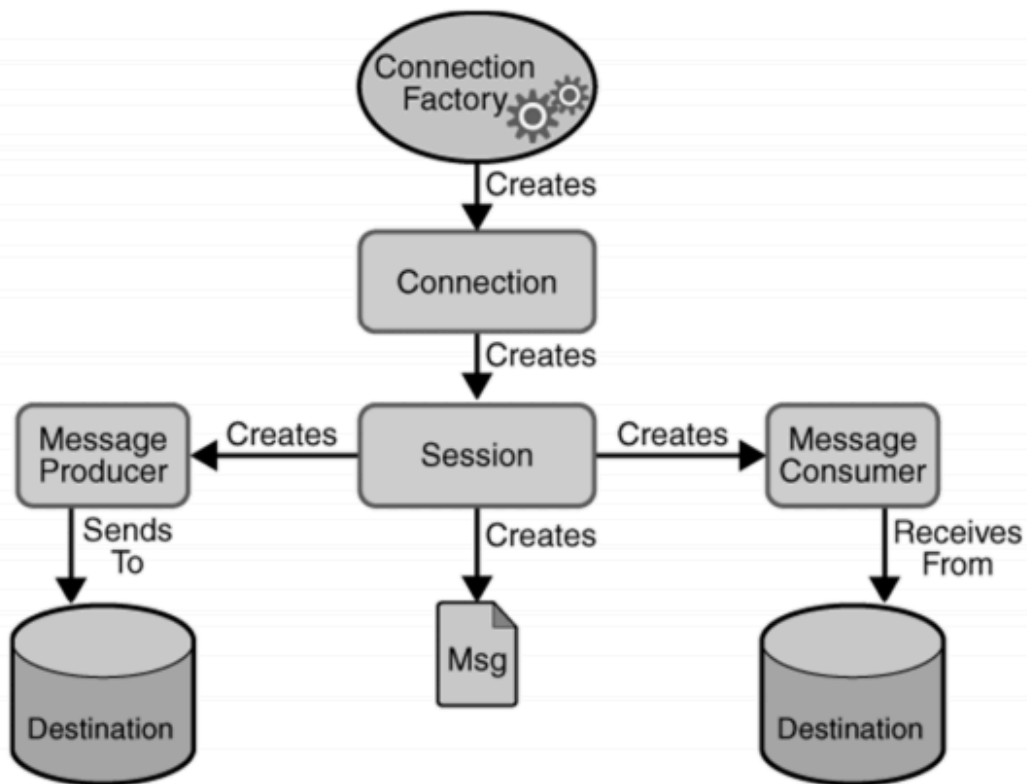
- Każda wiadomość może mieć wielu konsumentów
- Wiadomości są dostarczane do wszystkich aktualnych subskrybentów a następnie są usuwane
- Nie można odczytać wiadomości sprzed subskrypcji
- Trwała subskrypcja (ang. durable subscription) – odbiór wiadomości z okresu nieaktywności klienta



Model odbioru (konsumpcji) wiadomości

- Konsumpcja synchroniczna – blokująca metoda `receive()` z (ewentualnym) ograniczeniem czasowym
- Konsumpcja asynchroniczna – odbiornik wiadomości (ang. message listener) – asynchroniczne wywołanie metody `onMessage()`

Model programistyczny JMS



Pokazane są mniej więcej etapy:

- Tworzymy fabrykę połączeń, tworzymy połączenie z dostawcą używając funkcji z fabryki połączeń
- Jak już mamy połączenie możemy utworzyć sesję (sesja zazwyczaj jest pod konkretny wątek)
- Sesja tworzy komunikaty (tworzymy sobie obiekt Msg w ramach sesji), obiekt konsumenta oraz producenta

Fabryki połączeń

- Ogólna: `ConnectionFactory` (interfejs bazowy dla poniższych)
- Dla kolejek: `QueueConnectionFactory`
- Dla tematów: `TopicConnectionFactory`

Destinations

- queue w przypadku komunikacji punkt-punkt
- topic w przypadku modelu subskrypcji

```
@Resource(mappedName="jms/Queue")  
private static Queue queue;
```

```
@Resource(mappedName="jms/Topic")  
private static Topic topic;
```

Połączenie

- Reprezentacja wirtualnego połączenia z dostawcą JMS
- Połączenie jest potrzebne do zainicjowania sesji
- Połączenie musi być **jawnie zamknięte** na końcu aplikacji
- (zwolnienie zasobów, m.in. otwartych sesji)
- Rozpoczęcie odbioru wiadomości: metoda start()
- Wstrzymanie odbioru wiadomości: metoda stop()

```
Connection connection = connFactory.createConnection();  
connection.start();  
...  
connection.stop();
```

Sesje

- Jednowątkowy kontekst do tworzenia i odbioru wiadomości
- Sesje tworzą:
 - wiadomości
 - producentów (nadawców) wiadomości
 - konsumentów (odbiorców) wiadomości

```
Session session = connection.createSession(false,  
Session.AUTO_ACKNOWLEDGE);
```

- Pierwszy argument wskazuje czy ma być tworzona transakcja

NO I JUŻ MI SIĘ NIE CHCE DALEJ TEGO OGARNIAĆ...

***** Od Roberta *****

2020-05-05

Zdalne wywołania procedur - RPC

http://www.cs.put.poznan.pl/dwawrzyniak/dokuwiki/lib/exe/fetch.php?media=slices:rpc_2s.pdf

client stub/proxy - namiastka klienta, imituje procedurę zdalną (wyślij żądanie -> odbierz odpowiedź)

- przeźroczystość dostępu - ukrycie komunikacji sieciowej przed aplikacją przez odpowiednio opakowane funkcji komunikacyjne
 - stub klienta - procedura lokalna odpowiedzialna za przesłanie danych do serwera i odebranie wyników
 - stub serwera - udostępnienie aplikacji po stronie serwera procedury lokalnej odpowiedzialnej za odebranie identyfikatora procedury zdalnej, parametrów oraz odesłanie wyników lub zgłoszenie wyjątku

3 warstwy:

- select
- chan
- blast - podział na mniejsze kawałki, przesłanie, złożenie

przekazywanie parametrów/wartości

- przez wartość - kopia parametru
- przez wskaźnik/referencję - zapobiega kopiowaniu dużych struktur, wskazanie na przestrzeń adresową
- przez kopiowanie i odtwarzanie - kopiowanie od klienta do serwera, operacja na zdalnej stronie

format kanoniczny - do przesyłania danych między klientami, a serwerem

problem przetwarzania struktur złożonych - czy po przetoczeniu otrzymamy taką strukturę obiektów? wiele strzałek do jednego obiektu - dla każdej strzałki nowy obiekt (XDR)

gwarancje wykonania

- semantyka co najmniej raz - po uzyskaniu odpowiedzi od serwera, klient ma pewność, że wywołana procedura wykonała się co najmniej raz
 - nie odpuszczamy?

- akceptujemy błąd? - nie wiemy nic - mogło się nie udać? mogło zostać wykonane wiele razy?
- semantyka co najwyżej raz - po uzyskaniu odpowiedzi od serwera, klient ma pewność, że wywołana procedura wykonała się dokładnie raz
 - więcej niż raz się nie wykona - to wiemy
 - nie mamy procedur idempotentnych? gwarancja, że nie wykona się coś 2 razy więcej
- semantyka dokładnie raz - niemożliwa do uzyskania, jeśli system jest narażony na awarię (system synchroniczny - modele teoretyczne)
- semantyka ewentualnie - brak żadnej gwarancji

dopuszczamy awaryjność - co wiemy po awarii?

- jeśli słyszymy dokładnie raz, to w przypadku awaryjności najczęściej to jest co najwyżej raz

wiązanie klient-serwer

- statyczne - stałe ip:port
- dynamiczne - klient uzyskuje adres serwera za pośrednictwem łącznika

procedura wiązania

- serwer rejestruje się w łączniku
- klient odpytuje łącznik - ma informację o serwerze
- później realizacja komunikacji między klientem, a serwerem
- jednorazowe wywołanie procedury - klient zwraca się do łącznika, który od razu wysyła komunikat do serwera (łącznik staje się proxy)

warianty użycia RPC

- wywołanie asynchroniczne - klient nie czeka na wynik przetwarzania procedury zdalnej
- wywołanie zwrotne - klient udostępnia zdalną procedurę po swojej stronie, przekazuje serwerowi informację o jej wykonaniu (zamiana ról klient-serwer)

2020-05-07

RPC - wiązanie dynamiczne klienta z serwerem

http://www.cs.put.poznan.pl/dwawrzyniak/dokuwiki/lib/exe/fetch.php?media=slices:rpc_2s.pdf

- serwerze rejestruje się do łącznika, klient odpytuje łącznik o id komunikacyjny, wywołanie procedury na serwerze
- jednorazowe - klient może się podłączyć do łącznika, który jednorazowo wykona jakieś polecenie na serwerze

protokoły obsługi komunikacji:

- BLAST - przesyłanie dużych komunikatów poprzez podział na mniejsze, transmisja chunków, złożenie po stronie odbiorcy
 - ostatni fragment - sygnał, że trzeba poskładać, bo mamy komplet
 - identyfikujemy czego brakuje - wysyłamy SSR o retransmisję
- CHAN - kanał - synchronizacja wymiany komunikatów
 - żądanie -> ACK / odpowiedź -> ACK
 - domniemane potwierdzenie - odpowiedź jest potwierdzeniem - szybkie, krótkie wykonania procedur
 - osierocenie obliczeń :(- poszło żądanie, ale serwer już nie słucha, klient też mógł przestać słuchać
 - próbkowanie - ping (klient) -> pong (server), co jakiś czas
 - heartbeat
- SELECT - przekazuje komunikaty do odpowiednich procesów

realizacja semantyki błędów

- nie można zlokalizować serwera - zgłoszenie wyjątku
- zaginione żądanie - retransmisja, po upływie określonego czasu
- zaginiona odpowiedź - retransmisja, trudno rozróżnić z zaginione żądanie
 - procedury idempotentne
 - numerowanie żądań i retransmisja
- awaria serwera
 - przed podjęciem realizacji - retransmisja
 - po wykonaniu - zgłoszenie wyjątku
- awaria klienta - osierocenie obliczeń

usuwanie osieroconych obliczeń

- eksterminacja - rejestrowanie działań klienta na nośniku niewrażliwym na awarie, usuwanie osieroconych obliczeń po restarcie klienta
- reinkarnacja - restart klienta - nowa epoka - usuwanie obliczeń związanych z poprzednią epoką
- łagodna reinkarnacja - usunięcie obliczeń rozpoczętych w starej epoce, dla których nie ma właściciela
- wygaśnięcie - przydział serwerowi czasu T na wykonanie procedury, jeśli serwer nie skończy przetwarzania w tym czasie - musi uzyskać kolejny przydział, pod warunkiem, że obliczenia nie zostały osierocone, po czasie T osierocone obliczenia same się zakończą, ping-pong - przydzielanie kolejnego kwantu czasu

wykorzystanie protokołów warstw niższych - protokół połączeniowy - rozwiązuje większość problemów

Sun RPC/XDR

http://www.cs.put.poznan.pl/dwawrzyniak/dokuwiki/lib/exe/fetch.php?media=slices:sun-rpc-xdr_2s.pdf

- identyfikacja procesu - nr programu, nr wersji, nr procedury
- oparty na TCP/IP - TCP/UDP

opis interfejsu w języku RPC (narzędzie rpcgen)

- definicja struktur danych
- specyfikacja procedur zdalnych -nr programu, nr wersji, nr procedury, typy argumentów, typy zwracanej wartości

```
// rkill.x - opis interfejsu
program REMKILL {
    version V1 {
        int rkill(int) = 1;
    } = 1;
} = 0x20000001;
```

```
struct rkill_params {
    int pid;
    int signum;
};

program REMOTE_KILL {
    version SPECIFIED_SIGNUM {
        int rkil(rkill_params) = 1;
    } = 2;
} = 0x20000001;
```

- posługiwanie się nie nazwami, a identyfikatorami procedur - można wykonać procedurę kogoś innego

clnt - klientka

svc - serwerowa

make -f Makefile.rkill - po usunięciu dowiązania do biblioteki nsl

komunikacja z łącznikiem - portmapper albo rpcbind

2020-05-12

Standard XDR

- opis struktur danych
- filtry - konwersja pomiędzy formatem maszyny, a formatem kanonicznym (w obu kierunkach)
- potoki - uchwyt na kontakt z miejscem przechowującym dane, działa na potoku, potok ustala kierunek, filtr działa w obu kierunkach

Potok:

- miejsce składowania danych XDR, uchwyt na miejsce przechowujące dane
- zapisujący (kodujący) / odczytujący (dekodujący) - potok decyduje czy wykonuje kodowanie, czy dekodowanie
- rodzaje:
 - potok na pliku
 - potok na obszarze pamięci (adres i rozmiar)
 - potok na strumieniu danych

Filtr:

- procedura konwersji, realizacja dostępu do danych w potoku - niezależnie od kierunku - samo ogarnie czy ma kodować, czy dekodować

```
// rkill.x - kompilowane z -N - nowsza wersja pozwala na więcej niż 1
argument
program REMKILL {
    version V2 {
        // only one argument allowed
        // int rkill(int pid, int sig) = 1;
    } = 1;
} = 0x200000001;
```

```
// rkill.x

struct RKILL_V2_PARAMS {
    int pid;
    byte sig;
}

program REMKILL {
    version V2 {
        int rkill(RKILL_V2_PARAMS) = 1;
    }
}
```

```
} = 1;  
} = 0x20000002;
```

w rkill_xdr.c - filtr xdr dla zdefiniowanej struktury

```
// struct.x  
union Un switch (byte b) {  
    case 1:  
        int i;  
    case 2:  
        float f;  
    default:  
        char c;  
};
```

2020-05-14

Java RMI

http://www.cs.put.poznan.pl/dwawrzyniak/dokuwiki/lib/exe/fetch.php?media=slices:rmi_2s.pdf

- serwer - mechanizm udostępniania obiektów
- zdalny obiekt w przestrzeni adresowej serwera
- forma dostępu - wywoływanie metod zdalnych wywiedzionych z `java.rmi.Remote` - przekazywanie przez referencję
- interfejs traktujemy jako typ danych
- informacja o typie obiektu - w czasie kompilacji
- serializacja obiektu - implementacja interfejsu `java.io.Serializable` - kopia obiektu od klienta do serwera
- RMI - bazuje na TCP

tworzenie środowiska:

- tworzenie interfejsu pochodnego od `Remote`
- zdefiniowanie klasy wywiedzionej z `java.rmi.server.UnicastRemoteObject` implementujący interfejs pochodny od `Remote`

uruchomienie `rmiregistry` - uruchomienie rejestru/łącznika

rejestr - obiekt wiążący nazwę z zdalnymi obiektami - `rmiregistry <port>`

klasa LocateRegistry - umożliwia tworzenie rejestru (fabryka rejestrów), Registry - klasa-obiekt rejestru

- rejestr zewnętrzny - AlreadyBoundException - server został zabity, ale obiekt pozostał jeszcze w rejestrze
- rejestr w serwerze - zapobiega ^, czyszczenie rejestru obiektów po zabiciu serwera

2020-05-21

Ada :(

- <http://www.cs.put.poznan.pl/dwawrzyniak/dokuwiki/lib/exe/fetch.php?media=slides:ada95-4-cpp.pdf>
- <http://www.cs.put.poznan.pl/dwawrzyniak/dokuwiki/lib/exe/fetch.php?media=slides:ada95-4-cpp-packages.pdf>
- <http://www.cs.put.poznan.pl/dwawrzyniak/dokuwiki/lib/exe/fetch.php?media=slides:ada95-pointers.pdf>
- <http://www.cs.put.poznan.pl/dwawrzyniak/dokuwiki/lib/exe/fetch.php?media=slides:ada95-wsplobieznosc.pdf>
- <http://www.cs.put.poznan.pl/dwawrzyniak/dokuwiki/lib/exe/fetch.php?media=slides:ada95-distcomp.pdf>

```
-- hello.adb
with Ada.Text_IO;

procedure Hello is
  -- definicja zmiennych
  i: Integer := 3;
  tab: rray(1..10) of Integer := (6,1,3,8,2,10, others => 0);

  type Integer_Array is array(1..10) of Integer;
  procedure Bubble_Sort(tab: in out Integer_Array) is
  begin
    null
  end

begin
  for j in 1..10 loop
    Ada.Text_IO.Put_line("Hello " & Integer'Image(i));
    i := i+1;
  end loop;

  while i < 25 loop
    Put_line(Integer'Image(1));
    i := i+1;
```

```
    if i < 15 then
        exit;
    end if;
end loop;
end Hello;
```

2020-06-02

- semafor ogólny
- semafor uogólniony

P - probieren - próbować - opuszczanie

V - ver...

```
-- obiekt chroniony
protected Semaphore is
    entry P(value: in Positive);
    entry V(value: in Positive);
private
    entry P_Wait(); -- wejście prywatne, osiągalne tylko z P i V
    sem_value: Integer
end Semaphore;

protected body Semaphore is
    entry P(value: in Positive) when sem_v > 0 and P_Wait'count = 0 is
    begin
        sem_v := sem_v - value;
        if sem_v < 0 then
            requeue P_Wait; -- rekolejkowanie
        end
    end P;

    entry P_Wait() when sem_v >= 0 is
    begin
        null;
    end P;

    procedure V(value: in Positive) is
    begin
        sem_v := sem_v + value;
    end V;
private
    sem_value: Integer
```

```
end Semaphore;
```

- warunek uwzględniający wartość parametru, przyjmujemy to w jednym wejściu, rekolejkujemy dalej, gdzie sprawdzamy czy warunek jest spełniony
- preferencja sprawiedliwości - FIFO
- preferencja przepustowości - jeśli wartość mniejsza, ale wejdzie, to może wyprzedzić inne żądanie opuszczenia, które pojawiło się wcześniej

Polyorb - object request broker

- węzeł - jednostka w modelu przetwarzania rozproszonego, na którym uruchomiony jest program
- partycja - jednostki tworzące program rozproszony, umożliwiają komunikację rozproszoną między węzłami
 - aktywne - wykonują przetwarzanie, nie mogą w pasywnych, docelowo proces
 - pasywne - tylko jednostki Pure lub Shared_Passive
- Pure - jednostka opracowana, nie zmienia się w trakcie wykonywania programu - nie trzyma stanu, stateless
- Shared_Passive - umożliwia współdzielenie danych i programów, przez partycje aktywne - obiekty chronione, ale bez wejść, tylko procedury i funkcje
- Remote_Types - podobna do Pure, dopuszcza zdalne typy wskaźnikowe - wskaźniki na zdalne obiekty
- Remote_Call_Interface - zawierająca definicję interfejsu do komunikacji pomiędzy aktywnymi partycjami

2020-06-16

kolejkowanie komunikatów i przestrzeń krotek

- skrzynka, bufor, kolejka
- wiadomość - porta danych + dodatkowe właściwości
- kolejka - temat (kolejka dla wielu wiad.)
- komunikacja nieustanna - mechanizm do utrzymywania kolejek - broker

JMS

- komunikacja - nieustanna, niezawodna, asynchroniczna
- modele komunikacji - point-to-point/pub-sub
- niezawodność - utrwalanie komunikatów na nośniku?

point-to-point

- wiadomości w kolejce do czasu odbioru (odbiorom jest potwierdzenie odebrania) lub przedawnienia (komunikat ma time to live)
- każda wiadomość ma 1 konsumenta

pub-sub

- topic - jako zestaw kolejek
- 1 wiadomość ma wiele konsumentów
- wiadomości dostarczane do wszystkich aktualnych subów, następnie usuwane
- nie można odczytać wiadomości sprzed subskrypcji
- trwała subskrypcja - odbiór wiadomości z zakresu nieaktywności klienta
- komunikacja synchroniczna (receive()), asynchroniczna (message listener - onMessage())
- connection factory -> connection -> session (sesja - kontekst komunikacji dla 1 wątku) -> sesja produkuje komunikaty
- destinations

session.createProducer(queue) - wiązanie producenta z kolejką

wiadomość składa się z nagłówka, właściwości (id, odbiorca, timestamp, priorytet, typ), ciało wiadomości

przeglądanie zawartości kolejki

niezawodność komunikacji poprzez

- potwierdzenia wiadomości
- trwałość komunikatów
- priorytety komunikatów
- czas życia komunikatów
- tymczasowe kolejki
- sesja transakcyjna - commit (odebrane komunikaty są potwierdzane) / rollback (nie wysyłamy acków - wiadomości są dostarczane ponownie)
- wiadomości asynchroniczne - tylko auto ack
- priorytet - określa preferencje i nie decyduje o bezwzględnej kolejności dostarczania wiadomości
- ttl = 0 - brak przedawnienia
- subskrypcja trwała - nawet jak klient się zatrzyma i wstrzyma - to wszystkie komunikaty będą dla niego przechowane, dopóki nie odsubskrybuje
- transakcyjność
 - commit - wysłanie wyprodukowanych i potwierdzenie odebranych - wysłanie po potwierdzeniu (uwaga na zakleszczenia)
 - rollback - usunięcie wyprodukowanych i ponowne dostarczenie odebranych (z pominięciem przedawnionych)
- kolejki "tymczasowe" - kolejka prywatna, w ramach sesji, można dołączyć ją do komunikatu - na tą kolejkę ma iść odpowiedź, dostępna tylko w ramach tej sesji - odpowiedź na żądanie z kolejką zwrotną

przestrzeń krotek (default Linda)

- miejsce niezależne od komunikujących się procesów
- luźne powiązanie komunikujących się procesów - adresacja komunikatów - asocjacyjna

krotka - uporządkowana kolekcja danych określonego typu

- przestrzeń krotek - wspólne miejsce dostępna dla kooperujących procesów, gdzie gromadzone są krotki
- interfejs
 - output - umieszczanie krotki (operacja blokująca)
 - input - pobieranie krotki (operacja blokująca)
 - read - odczytanie krotki bez pobierania
 - try_input, try_read - nieblokujące wersje input/read

Output(4,1,"Test") - umieszczenie krotki w przestrzeni - jej parametrami są wartości atrybutów

Input(4,1,text:String) - odebranie (usunięcie) krotki z pamięci, na podstawie atrybutów które są określone, pozostałe zostaną dopasowane do typów zmiennych, jeśli więcej krotek z tymi atrybutami - wyciągamy dowolną krotkę

Output(1,"Anyone?") - krotka bez pewnych atrybutów - może zostać pobrana z dowolną wartością tego atrybutu

- zbiór, a nie kolejka - krotki nie są uporządkowane i mogą być odbierane w innej kolejności niż zostały umieszczane

Javaspaces

- obiekty implementujące interfejs Entry, pola muszą być publicznymi obiektami - puste referencje (null'e) - brak parametrów
- krotka ma ograniczony czas życia (czas dzierżawy)
- write, take, read, takeIfExists, readIfExists

Java – ZADANIA

Problem Konsumenta–Producenta (elementarna synchronizacja wątków #synchronized)

```
public class Consumer implements Runnable {
    private Buffer buffer;
    private String id;

    public Consumer(Buffer buffer, String id) {
        this.buffer = buffer;
        this.id = id;
    }

    @Override
    public void run() {
        int i=0;
        while(i<100)
        {
            synchronized (buffer)
            {
                if(!buffer.isEmpty())
                {
                    System.out.println(id + " retrieved " + buffer.get());
                    i++;
                }
                else
                {
                    try {
                        buffer.wait();
                    } catch (InterruptedException e) {}
                }
            }
        }
        System.out.println(id + " is done");
    }
}
```

```
public class Producer implements Runnable {
    private Buffer buffer;
    private String id;

    public Producer(Buffer buffer, String id) {
        this.buffer = buffer;
        this.id = id;
    }

    @Override
    public void run() {
        int i=0;
        while(i<100)
        {
            synchronized (buffer)
            {
                if(buffer.hasSpace())
                {
                    buffer.put(i);
                    System.out.println(id + " produced " + i);
                    i++;
                    buffer.notifyAll();
                }
            }
        }
        System.out.println(id + " is done");
    }
}
```

Problem Konsumenta–Producenta (Semafor)

```

public class BufferSemaphore implements Buffer {

    private int max;
    private ArrayList<Integer> buffer;
    private Semaphore semaphore;
    public BufferSemaphore(int max)
    {
        this.max = max;
        this.buffer = new ArrayList<>();
        semaphore = new Semaphore(1,true);
    }
    @Override
    public void put(int v) {
        while(buffer.size()>=max || !semaphore.tryAcquire())
        {
            try {
                sleep(10);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
        buffer.add(0,v);
        semaphore.release();
    }

    @Override
    public int get() {
        int x = 0;
        while(buffer.size()==0 || !semaphore.tryAcquire())
        {
            try {
                sleep(10);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
        x = buffer.get(buffer.size()-1);
        buffer.remove(buffer.size()-1);
        semaphore.release();
        return x;
    }
}

```

Producent–Konsument (Lock)

```
private int max;

private ArrayList<Integer> buffer;
private Lock lock;
private Condition conditionFull;
private Condition conditionEmpty;

public BufferLock(int max) {
    this.max = max;
    this.buffer = new ArrayList<>();
    this.lock = new ReentrantLock();
    this.conditionFull = lock.newCondition();
    this.conditionEmpty = lock.newCondition();
}
```

```
@Override
public void put(int v) {
    lock.lock();
    try{
        while(buffer.size()>=max){
            conditionFull.await();
        }
        buffer.add(v);
        conditionEmpty.signalAll();
    }catch(InterruptedException e){
        e.printStackTrace();
    }
    finally{
        lock.unlock();
    }
}

@Override
public int get() {
    int x = -1;
    lock.lock();
    try
    {
        while(buffer.size()==0)
        {
            conditionEmpty.await();
        }
        x=buffer.get(buffer.size()-1);
        buffer.remove(buffer.size()-1);
        conditionFull.signalAll();
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
    finally {
        lock.unlock();
    }
    return x;
}
```

ADA

Semafor ogólny

```
with Ada.Text_IO; use Ada.Text_IO;

procedure semafor_ogolny is

type Index is mod 5;
type Buffer_Array is array (Index) of Integer;

task type Producent;
task type Konsument;
task Buffer is
  entry Put(v: in Integer);
  entry Get(v: out Integer);
end Buffer;

protected Semaphore is
  procedure P;
  entry V;
private
  Counter : Integer := 0;
end Semaphore;

protected body Semaphore is
  procedure P is
  begin
    Counter := Counter + 1;
  end P;
  entry V when Counter > 0 is
  begin
    Counter := Counter - 1;
  end V;
end Semaphore;

protected Buffer_Obj is
  entry Put(v: in Integer);
```

```

    entry Get(v: out Integer);
private
    buf: Buffer_Array;
    put_pos, get_pos: Index := 0;
    liczba: Integer range 0..5 := 0;
end Buffer_Obj;

protected body Buffer_Obj is
    entry Put(v: in Integer) when liczba < 5 is
    begin
        buf(put_pos) := v;
        put_pos := put_pos + 1;
        liczba := liczba + 1;
    end Put;
    entry Get(v: out Integer) when liczba > 0 is
    begin
        v := buf(get_pos);
        get_pos := get_pos + 1;
        liczba := liczba - 1;
    end Get;
end Buffer_Obj;

task body Producent is
begin
    for i in 1..20 loop
        Semaphore.P;
        Put_line("Wstawiono: " & Integer'Image(i) );
        Buffer_Obj.Put(i);
        --delay 0.5;
        Semaphore.V;
    end loop;
end Producent;

task body Konsument is
    x: Integer;
begin
    for i in 1..20 loop
        Semaphore.P;
        Buffer_Obj.Get(x);
        Put_line("Odebrano: " & Integer'Image(x) );
        delay 0.2;
    end loop;
end Konsument;

```

```

    Semaphore.V;
  end loop;
end Konsument;

task body Buffer is
  buf: array (Index) of Integer;
  put_pos, get_pos: Index := 0;
  liczba: Integer range 0..5 := 0;
begin
  loop
    select
      when liczba < 5 => accept Put(v: in Integer) do
        buf(put_pos) := v;
        put_pos := put_pos + 1;
        liczba := liczba + 1;
      end Put;
    or
      when liczba > 0 => accept Get(v: out Integer) do
        v := buf(get_pos);
        get_pos := get_pos + 1;
        liczba := liczba - 1;
      end Get;
    end select;
  end loop;
end Buffer;

producenty: array (1..3) of Producent;
konsumenty: array (1..3) of Konsument;
begin
  null;
end semafor_ogolny;

```

Semafor uogólniony z przepustowością

```

with Ada.Text_IO; use Ada.Text_IO;

```

procedure sem_counting is

-- CountingSemaphore

protected type CountingSemaphore(Initial_Count: Integer := 1) is

entry P(C: in Integer);

procedure V(C: in integer);

private

Count : Integer := 1;

Ready : Boolean := False;

Check_Count : Integer := 0;

entry P_Again(C: in Integer);

end CountingSemaphore;

protected body CountingSemaphore is

entry P(C: in Integer) when True is

begin

if Count >= C then

Count := Count - C;

else

requeue P_Again;

end if;

end P;

procedure V(C: in Integer) is

begin

Count := Count + C;

Check_Count := P_Again'Count;

if Check_Count > 0 then

Ready := True;

end if;

end V;

entry P_Again(C: in Integer) when Ready is

begin

Check_Count := Check_Count - 1;

if Check_Count = 0 then

Ready := False;

end if;

if Count >= C then

Count := Count - C;

else

```

        requeue P_Again;
    end if;
end P_Again;
end CountingSemaphore;

S: CountingSemaphore(1);

-- Shared Variable
X : Integer;

task type Producent;
task type Konsument;

task body Producent is
begin
    for i in 1..20 loop
        S.P(1);
        X := i;
        Put_line("Wstawiono: " & Integer'Image(X));
        S.V(1);
        delay 0.5;
    end loop;
end Producent;

task body Konsument is
begin
    for i in 1..20 loop
        S.P(1);
        Put_line("Odebrano: " & Integer'Image(X));
        S.V(1);
        delay 0.5;
    end loop;
end Konsument;

producenty: array (1..3) of Producent;
konsumenty: array (1..3) of Konsument;

begin
    null;
end sem_counting;

```

```
-- preferencja sprawiedliwości (fairness) - zadania w kolejce, nei
przepuszczamy, anwet jeśli pojawi się jakieś zadanie, któe mogłoby przejść
-- preferencja przepustowości - jak pojawi się jakieś zadanie, które ma
semafor spełnia, to można wejść do sekcji, mimo tego, że jakieś inne czekają -
pierwsze zadanie takie, które spełnia wymagania semafora
```

Bariera

```
protected Barrier is
  entry Await;
private
  entry innerAwait;
  count : integer := N;
end Barrier;
protected body Barrier is
  entry Await when true is
begin
  count := count - 1;
  requeue innerAwait;
end Await;
entry innerAwait when count = 0 is
begin
  null;
end innerAwait;

end Barrier;
```

JAVA:

```
public class Barrier {
  private int count;
  public Barrier(int noThreads) {
    super();
    this.count = noThreads;
  }
}
```

```

public synchronized void await() throws InterruptedException {
    count--;
    while (count > 0) {
        this.wait();
    }
    this.notifyAll();
}
}

```

----- (PRAWIDŁOWA WERSJA) :) Będzie fajnie :) /// NIE JEST
PRAWIDŁOWA

```

public class Barrier {
    private int count;
    private int barrierStrength;
    public Barrier(int noThreads) {
        super();
        this.count = noThreads;
        this.barrierStrength = noThreads;
    }

    public synchronized void await(int weight) throws InterruptedException {
        this.count = this.count - weight;
        while (this.count > 0) {
            this.wait();
        }
        this.notifyAll();
        this.count = this.barrierStrength;
    }
}

```

ADA (bariera z wytrzymałością) // To jak to powinno być ?

protected Barrier is

 entry Await (weight : in Integer);

private

 entry innerAwait;

 count : integer := 10;

 barrierStrength : integer := 10;

end Barrier;

protected body Barrier is

 entry Await(weight : in Integer) when true is

begin

 count := count - weight;

 requeue innerAwait;

end Await;

entry innerAwait when count <= 0 is

begin

 count := barrierStrength;

end innerAwait;

end Barrier;

```
protected Barrier is
    entry Await (weight : in Integer);
private
    entry innerAwait;
    count : integer := 10;
    barrierStrength : integer := 10;
end Barrier;
protected body Barrier is
    entry Await(weight : in Integer) when count > 0 is
begin
    count := count - weight;
    requeue innerAwait;
end Await;
entry innerAwait when count <= 0 is
begin
```

```

    if innerAwait'count = 0 then
        count := barrierStrength;
    end if;
end innerAwait;

end Barrier;

```

Proszę przedstawić realizację mechanizmu synchronizacji opartego na obiekcie chronionym w środowisku Ada-95, udostępniającego dwie operacje: wait oraz signal, których działanie jest następujące:

- wait wywoływane jest z parametrem, który jest priorytetem, blokuje wywołujący je proces (wątek, zadanie) do momentu otrzymania sygnału poprzez wywołanie signal,
- signal odblokowuje proces (wątek, zadanie) o najwyższym priorytecie, a w przypadku braku oczekujących zadań nie ma żadnego skutku dla stanu systemu.

```

protected Synchr is
    entry Signal ();
    entry Wait (IN priority : integer);
    maxPriority = 0 : Integer
private
    entry Queue ();
    isSignal : boolean := False;
    isSignalRequeue : boolean := False;
end Synchr;
protected body Synchr is
entry Signal() is
begin
    isSignal := True;
    isSignalRequeue := True;
end Signal;
entry Wait (IN priority : integer) when isSignal is

```

```

begin
    if maxPriority < priority then
        maxPriority := priority;
    end if;
    requeue Queue
end Synchr ;
entry Queue(IN priority: integer) when isSignalRequeue is
begin
    if Queue'count = 0 then
        maxPriority := 0;
        isSignalRequeue := False;
    end if;
    if !isSignal or priority < maxPriority then
        requeue Wait;
    end if;
    if maxPriority = priority and isSignal then
        isSignal := False;
    end if;
end Queue

```

```

with Ada.Text_IO; use Ada.Text_IO;

procedure Egzamin is

    protected Watki is
        -- Operations go here (only subprograms)
        entry Wait(v: in Integer);
        entry Signal;
        entry WaitAgain(v: in Integer);
    private

```

```

MaxPriority: Integer := 0;
NextMaxPriority: Integer := 0;
MaxReleased : Boolean := false;
Released: Integer := 0;
end Watki;
protected body Watki is
    entry Wait(v: in Integer) when 1=1 is
    begin
        Put ("weszło ");
        Put_line (Integer'Image (v));
        if MaxPriority < v then
            MaxPriority := v;
        end if;
        requeue WaitAgain;
    end Wait;

    entry Signal when Released=0 is
    begin
        Released := WaitAgain'count;
        MaxReleased := false;
        NextMaxPriority := 0;
    end Signal;

    entry WaitAgain(v: in Integer) when Released > 0 is
        againToQue:Boolean := true;
    begin
        Released := Released - 1;
        if (MaxReleased = false and v = MaxPriority) then
            MaxReleased := true;
            againToQue := false;
            Put_line (Integer'Image (v));
        elsif NextMaxPriority < v then
            NextMaxPriority := v;
        end if;

        if Released = 0 then
            MaxPriority := NextMaxPriority;
        end if;
    end
end Watki;

```

```
        if againToQue = true then
            requeue WaitAgain;
        end if;
    end WaitAgain;
end Watki;

task z1;
task body z1 is
begin
    Watki.Wait(3);
    Put_line ("wyszedłem 3");
end z1;

task z2;
task body z2 is
begin
    Watki.Wait(4);
    Put_line ("wyszedłem 4");
end z2;

task z3;
task body z3 is
begin
    Watki.Wait(5);
    Put_line ("wyszedłem 5");
end z3;

task z4;
task body z4 is
begin
    Watki.Wait(5);
    Put_line ("wyszedłem 5");
end z4;

task type Z5;
task body Z5 is
begin
    for i in 1..3 loop
        Put_line("Sygnał");
```

```

        Watki.Signal;
    end loop;
end Z5;

zadanie: Z5;

begin
    Put_line("start");
end Egzamin;

```

// Bariera Java:

```

public class Barrier {
    private final int strength;
    private int currentBreak = 0;

    public Barrier(int strength) {
        this.strength = strength;
    }

    public synchronized void breakthrough(int
breakCount) throws InterruptedException {
        currentBreak += breakCount;
        System.out.println(currentBreak);
        if (currentBreak < strength) {
            wait();
        } else {
            currentBreak = 0;
            notifyAll();
        }
    }
}

```

```
} [REDACTED]
```